OBJECT DETECTION FOR VISUALLY IMPAIRED PEOPLE

Project Report Submitted in Partial Fulfillment of the Requirements for the Award of the Degree of
BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

By

B. SNEHITA	**19501A0528**
G.SRI SRAVANI	**19501A0565**
B. YEGNESH SAI	**19501A0523**
B. SUDEEP	**19501A0525**
B. CHARAN KARTHIK	**19501A0519**

Under the Guidance of

Mr . A. PRASHANT

Assistant Professor

Department of Computer Science and Engineering

PRASAD V POTLURI SIDDHARTHA INSTITUTE OF TECHNOLOGY

(Permanently affiliated to JNTUK: Kakinada, Approved by AICTE)

(An NBA & NAAC accredited and ISO 9001:2015 certified institution)

Kanuru, Vijayawada - 520007

2022-2023

PRASAD V. POTLURI SIDDHARTHA INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the final project entitled **"OBJECT DETECTION FOR VISUALLY IMPAIRED PEOPLE"** being submitted by **B.SNEHITA (19501A0528), G.SRI SRAVANI (19501A0565), B.YEGNESH SAI (19501A0523), B.SUDEEP (19501A0525), B. CHARAN KARTHIK (19501A0519)** in partial fulfilment for the award of the Degree of Bachelor of Technology in Computer Science and Engineering to the Jawaharlal Nehru Technological University, Kakinada is a record of Bonafide work carried out under my guidance and supervision.

Guide Signature with Date

Mr.A.PRASHANT
Assistant Professor,
Dept. of CSE,
Prasad V Potluri Siddhartha Institute of Technology, Vijayawada

Head of the Department Signature

Dr. A. JAYALAKSHMI
Professor & Head,
Dept. of CSE,
Prasad V Potluri Siddhartha Institute of Technology, Vijayawada

Signature of the External

ACKNOWLEDGEMENT

Behind every achievement lies an unfathomable sea of gratitude to those who activated it, without whom it would ever have come into existence. First and foremost, we would like to convey our heart full thanks to the Almighty for the blessings on us to carry out this project work without any disruption. To them ,we lay the words of gratitude imprinted on us.We would like to thank our respected Principal, **Dr. K. SIVAJI BABU** and **Dr. A. JAYA LAKSHMI**, Head of the Department, Computer Science and Engineering, for their support throughout our project.

It is our sincere obligation to thank our guide, **Mr. A. Prashant,** Assistant Professor, Computer Science and Engineering Department, for his timely valuable guidance and suggestions which helped to bring out this project successfully. His wise approach made us learn the minute details of the project. His patient guidance paved the way for completing our project with a sense of satisfaction and pleasure.

We are thankful to our project coordinator's **Ms. A. Ramana Lakshmi** and **Mr. Dr. Ravuri Daniel** for their valuable guidance which helped us to bring this project successfully.

Finally, we would like to convey our heart full thanks to our Technical Staff for their guidance and support in every step of this project. We convey our sincere thanks to all the faculty and friends who directly or indirectly helped us for the successful completion of this project.

B. SNEHITA **19501A0528**
G. SRI SRAVANI **19501A0565**
B. YEGNESH SAI **19501A0523**
B. SUDEEP **19501A0525**
B. CHARAN KARTHIK **19501A0519**

DECLARATION

We hereby declare that the Major Project entitled "**OBJECT DETECTION FOR VISUALLY IMPAIRED PEOPLE**" submitted for the B. Tech Degree is our original work and the dissertation has not formed the basis for the award of any degree, associateship, fellowship, or any other similar titles.

Place: Vijayawada

Date:

B. SNEHITA	**19501A0528**
G. SRI SRAVANI	**19501A0565**
B. YEGNESH SAI	**19501A0523**
B. SUDEEP	**19501A0525**
B. CHARAN KARTHIK	**19501A0519**

ABSTRACT

In recent graphs, Vision Impairment is one of the top disabilities globally. Visually impaired people find it difficult to identify the objects around them using voice commands. The voice will be taken with the help of microphone using a Python library known speech recognition. This project proposes an application for object detection to assist visually impaired people. This application uses deep learning algorithms for object detection and for finding object distance and direction. The model will be trained using CNN architecture with the help of predefined datasets from TensorFlow. The objects will be detected in the trained datasets. Object detection includes various processes like image segmentation and image recognition which help to increase the accuracy of the application. An application will be developed to integrate it with the camera and audio drivers. The output will be in the form of voice, specifying the detected objects and their positions through the device speaker . This project helps blind people get benefitted to perform their daily activities efficiently.

Keywords: Deep Learning, YOLOv3, TensorFlow, COCO datasets, Speech Recognition.

.

LIST OF FIGURES

FIGURES **PAGE NO**

Fig:2.1. System Architecture 17

Fig:2.2: System Architecture 19

Fig:2.3.1: Process Flow 21

Fig:2.3.2: Basic Flow of Algorithm 22

Fig:2.4: Basic flow of Algorithm 24

Fig:5.1: System Architecture 28

Fig:5.2.1. A:Usecase Diagram 31

Fig: 5.2.1.B: Sequence Diagram 32

Fig: 5.2.1.C: Class Diagram 33

Fig: 5.2.1.D: Activity Diagram 34

Fig: 5.2.1.E: State Chart Diagram 35

Fig: 5.3.1.1: Speech Recognition 36

Fig: 5.3.1.2: Classification vs Localization vs Detection 38

Fig: 5.3.1.3.1: Layers in YOLOv3 39

Fig: 5.3.1.3.2: Detecting objects at three scales 40

Fig: 5.3.1.4.1: Anchor Boxes 42

Fig: 5.3.1.5.1: Predicting Bounding boxes 44

Fig: 5.3.1.5.2: Predicting Bounding boxes in steps 45

Fig: 5.3.1.5.3: Predicting Bounding boxes in images 45

Fig: 5.3.1.6: Non-Max Suppression 46

Fig: 6.2.1.1: Weights of the model 54

Fig: 6.2.1.2: Saved model.h5 55

Fig: 6.2.3.2: To display Direction 65

Fig: 6.2.3.3: Logical view of Direction for an object 66

Fig:7.2.2: Output for object detection in live video 72

Fig:7.2.4: Output for finding a object in live video 76

Fig 8.1: Output for detecting objects 77
Fig 8.2: Output for finding objects 77

LIST OF SYMBOLS

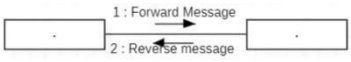
Name	Symbol	Description
Actor		The entities which interact with the system are termed as actors in a use case diagram
Communication		Communication between various use cases.
State	State	State of the process.
Initial State		Initial State of the object
Final State		Final State of the object
Control flow		Represents various control flow between the states.
Use-case		Interaction between the system and the external environment.
External entity		Represents external entities such as keyboard,sensors,etc.
Transition		Represents communication that occurs between processes.

TABLE OF CONTENTS

CONTENT	PAGE NO
Abstract	5
List of Figures	7
List of Symbols, Abbreviations	8
CHAPTER 1: INTRODUCTION	**11**
1.1. Introduction to project	11
1.2. Basic concepts	12
1.3. Statement of the problem and solution to the problem	14
1.4. Objectives	15
1.5. Significance of work	15
CHAPTER 2: BACKGROUND AND LITERATURE REVIEW	**16**
2.1. Moving Object Detection with Deep CNN[2019][1]	16
2.2. Object Detection and Recognition using TensorFlow for Blind People [2022][2]	19
2.3. Detecting objects and people and tracking movements in a video using TensorFlow and deep learning [2020]	21
2.4. An Object Detection Technique for Blind People in Real-Time Using Deep Neural Network [2019][4]	23
CHAPTER 3: SYSTEM ANALYSIS	**25**
3.1. Existing system	25
3.2. Proposed system	25
3.3. Feasibility study	25
3.3.1. Technical Feasibility	25
3.3.2. Operational Feasibility	26
3.3.3. Economical Feasibility	26

CHAPTER 4: SYSTEM REQUIRMENT SPECIFICATIONS 27
4.1. Functional requirements 27
4.2. Non-Functional requirements 27
CHAPTER 5: DESIGN AND METHODOLOGY OF PROPOSED MODEL 28
5.1. System Architecture and Model 28
5.2. System Design 29
5.2.1. Uml Diagrams 29
5.3. Methodology 36
5.3.1. Process Model/Methodology/Algorithm 36
5.3.2. Tools and Techniques 47
CHAPTER 6: IMPLEMENTATION OF BASE PAPER (CODING) 48
6.1. Modules 48
6.2. Description of sample code of each module 48
CHAPTER 7: TESTING 68
7.1. Testing strategies 68
7.2. Test cases 69
CHAPTER 8: RESULT 77
CHAPTER 9: CONCLUSION 78
9.1. Conclusion 78
9.2. Future work 78
REFERENCES & BIBILOGRAPHY 80
Appendix-A
A.1. Full Code
A.2. Screenshot

1.INTRODUCTION

1.1.Introduction to Project

Object detection is a computer vision and image processing technique that deals with detecting instances of semantic objects of a certain class (such as humans, buildings, or cars) in digital images and videos. Some of the real-time applications of object detection are self-driving cars, Face Detection, Pedestrian detection, crowd counting, etc.

Object detection has broken down into both machine-learning and deep-learning approaches. In Machine learning approaches, it uses computer vision techniques such as color histogram to identify groups of pixels, etc that belong to an object all the features are fed a regression model which helps to predict the object's location and its label. In deep learning approaches, it uses a convolution neural network to perform unsupervised object detection. The classification model has been fed by all the features. In an image or video, interested regions are selected and then classified using CNN.

In CNN, interested regions are selected which leads to running the model very slowly. In YOLO, there is no need to select the interested regions instead it predicts the classes and bounding boxes of an image or video on a single run and can detect multiple objects in a single neural network.

Object detection uses a sliding window technique for detecting objects. The main aim of object detection is to detect objects from an image or video. In this application, the most common objects are detected like buses, people, cars, trees, mobile, pens, etc. The YOLOv3 algorithm is used for detecting objects in this application. Concepts of object localization are used for detecting more objects in an image or video.

Object Detection: This is done by, Locating the presence of objects with a bounding box and types or classes of the located objects in an image. The Input is an image that consists of one or more objects, such as a photograph. The Output is One or more bounding and each bounding box's class label.

YOLO algorithm is faster than other classification algorithms because it predicts false positives in the background. YOLO trains its images with datasets like COCO, VOC and ImageNet datasets and YOLO does not test the degraded images. YOLO takes every image as a grid and applies image classification algorithms on each grid cell.

The YOLO algorithm is important due to its:

- Speed
- High Accuracy
- Learning capabilities

1.1.1. Basic Concepts

➢ Deep Learning

Deep Learning is a subset of Artificial Intelligence. As humans would be continually analyzing data, deep learning algorithms attempt to draw similar conclusions and it is implemented with the help of deep networks, which are nothing but neural networks with multiple hidden layers.

Deep-learning architectures such as deep neural networks, deep belief networks, deep reinforcement learning, recurrent neural networks, and convolutional neural networks have been applied to fields like computer vision, speech recognition, and natural language processing, where they have produced results comparable to and in some cases surpassing human expert performance.

A Convolutional Neural Network(CNN) is a deep-learning algorithm that can take an image as input with some weights and that will be given to the hidden layers of the network. Convolutional Neural Networks can learn filters/characteristics. After applying algorithms/frameworks of CNN such as Darknet, YOLO, Resnet50, etc. The output of the hidden layer is given as input to the hidden layer and then gives the output of the network through the output layer.

- **Python Language**

Python language is an obect-oriented programming language used to write code for data visualization, artificial intelligence and data science. Compared with other programming languages. Python is easier according to syntax and also has more built-in functions which makes writing code easier and also easy to understand for the user.

- **YOLO V3**

YOLOv3 is one of the versions of YOLO algorithms, used for real-time object detection in videos and images. YOLOv3's architecture allows it to predict at 3 different scales, that leads to extraction of features at layers 82, 94, and 106. It can classify 80 object categories.

- **Open CV**

Open CV is an open-source library that is used to solve problems of Computer vision and image processing and analysis. It can be used in AI and with its subsets. Nowadays Open CV is playing a crucial role in the real-time industry. Along with well-established companies like Google, Yahoo, Microsoft, Intel, IBM, Sony, Honda and Toyota that employ the library. There are many startups such as Applied Minds, VideoSurf and Zeitera that make extensive use of OpenCV

- **Speech Recognition**

Speech Recognition technology has grown exponentially in recent years, with numerous application in various industries. This sophisticated technology enables machines to understand and interpret human speech accurately. Speech recognition software can be used for a wide range of purposes, including communication aids for people with disabilities, virtual assistants, language translation services, automated customer service systems and much more . Speech recognition software works by processing audio signals from the spoken word and converting it into text or commands which are understood by computer programs. The process involves several steps that are carefully designed to ensure accuracy and reliability. First, the system captures an audio signal using a microphone or other input device; then it analyses the signal using complex algorithms to identify speech patterns

such as phonemes (the smallest unit of sound in language), words or phrases. Finally, the system employs natural language processing techniques to understand the meaning behind what was said.

1.2. Motivation

The motivation for developing this application is to provide an easier and more convenient way to make visually impaired people do their tasks. This application of object detection is introduced to focus on giving services to all visually impaired people and making their life become better. Everyone, no matter where they are, should do all their tasks efficiently. Hence we want the project to have a significant impact on society in such a way that visually impaired person can do their daily routines as visually paired people do.

1.3. Statement of the problem and Solution to the Problem

- **Problem Statement:**

Build an application for Object detection for Visually Impaired people using Deep Learning.

Millions of people over the world were having Visual Impairment. Visually Impaired people face difficulties such as navigating routes and finding things. For example, if a blind person wants to find a particular room number. As he/she was unfamiliar with the environment, it would be a difficult task. This makes people dependent on others. Also, it is difficult to cross the roads and to know whether the people were talking to them or not.

- **Solution to the Problem:**

Deep Learning technologies have been rapidly developed in recent years. To solve this problem, a smart application is developed which detects objects in videos or images with mobile using deep learning technologies. The model is done using YOLOv3 with Keras and then after training the model the objects will be predicted. Once the object Detection is completed, the user will be specified by the distance between the object and the person, and the direction where the object is there. In addition, to calculate distance and direction

of objects, this model also finds and detects a particular object until user gives a command like to stop detection or to stop finding object.

This project proposes a mobile application to make it easier for the user to use. With the help of a device camera, the objects will be captured from images and videos and will be detected and assist blind people.

1.4. Objectives:

The objectives of a project include the following:

- **General Objectives**
  - To make the application communicate with the user in a way that a visually impaired person can respond to.
  - Easy to use. (Doesn't require any technical knowledge)
  - The device needs to be portable.

- **Specific Objectives**
  - To detect objects for blind people with a camera.
  - To inform the user of potential hazards, stairs, and if a user is entering a crosswalk/street.
  - Use voice output to assist and navigate blind people.

1.5. Significance Of Work

The proposed model which when implemented in the real-world and have many areas of applications. It helps blind people to detect objects in surroundings. This helps to make their daily routines easily.

2. LITERATURE SURVEY

2.1. Moving Object Detection With Deep CNN [2019][1]

Real- time detection and identification of moving objects with high accuracy is very challenging work due to excessive amount of data this paper proposes an over framework that consists of a coarse-grained detection along with fine-grained detection. To solve the problem of noise induced objective fracture during the coarse-grained detection process ,we present a low complexity connected region detection algorithm to extract moving regions furthermore, in the fine-grained detection ,Deep convolutional neural networks are leveraged to detect more precise coordinates and identify the category of objects.

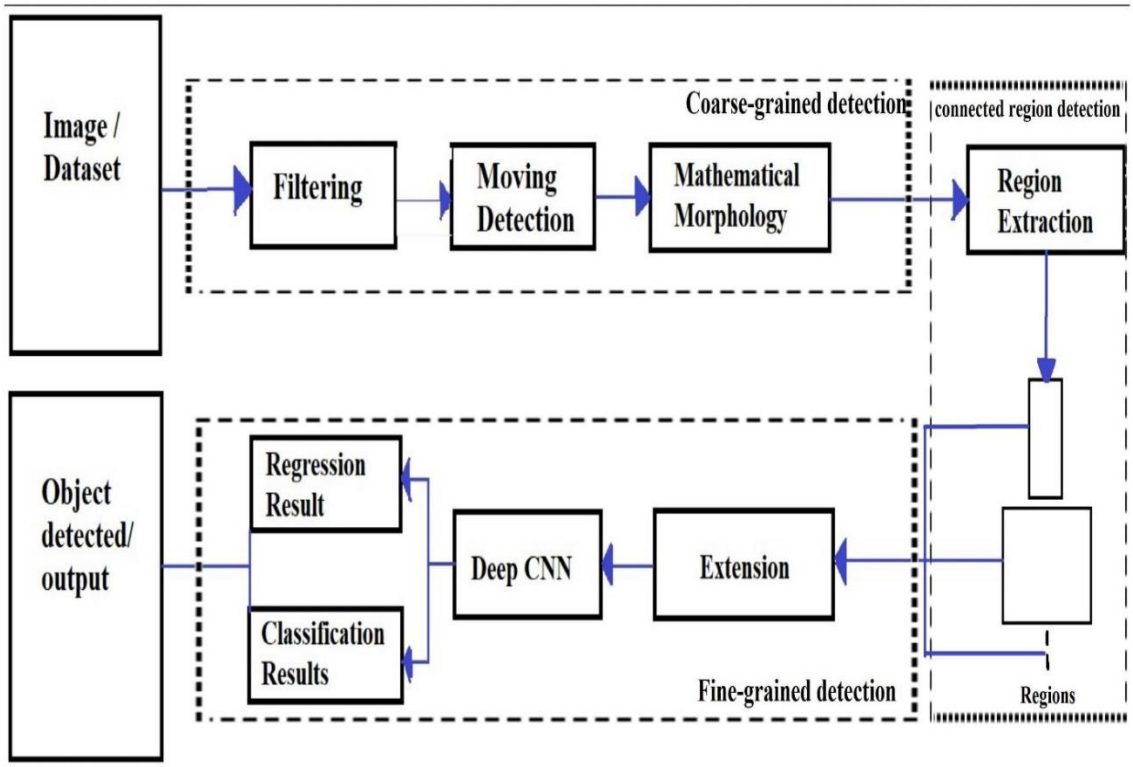
Proposed methods :

1. Coarse-gradient detection
2. Connected region detection
3. Fine-gradient detection

Further moving on to the working of the model , each video frame is divided into fore - background to detect moving objects according to the difference of pixel intensity or color distribution. The below figure [2.1] methods of artificial design for foreground extraction and it's framework schema.

A: Coarse-grained detection:

In order to get a high speed and accuracy detection on high resolution scenes, this paper propose a coarse-to-fine grained detection which combines moving detection and identification to get the category and bounding box of each moving object. Coarse-grained detection is performed firstly to obtain moving regions due to presence of complex scenes, local motion, illumination variation and so on, the regions obtained may be inaccurate such as containing non object or broken object[2.1].



2.1.System Architecture

To merge the regions that contain the fractured objects, we propose allow-complexity method to extract the connected regions. Then, by the fine-gradient detection with the deep CNN coordinates and categories of each moving object are obtained. Furthermore, in order to get faster speed, YoloV3 and its tiny version are modified for fine-gradient detection.

The innovation points proposed in this paper are summarized in the following:

1. course to- fine gradient moving object detection framework which combines moving detection with DCNNs is proposed.
2. an effective algorithm is proposed to detect the connected regions.
3. sure the tiny version of Yolo V3 is modified to make the direction faster.

B: Connected region detection:

Due to the noisy background, the same object may be segmented. To deal with this issue we propose an efficient region extraction algorithm. The connected-region detection takes

the image from coarse-radiation detection in the form of the object detected in white pixel and the filtering in the coarse-gradient detection is used to detect the noise and removes it and gives it as the output to this coarse-connected detection. The connected-region detection takes a black- and-white morphed image and filters it into the grid.

C: Fine-grained detection:

In fine-grained detection, objects occupy a large area of the region which is beneficial for object detection with DCNNs. The fine-grained detection is mainly used for sharp etching and removal of background noisy data by using the outputs obtained from the connected region reduction.

Compared with accuracy and speed of the network is more concerned issue taken here.For resolving it, the used YOLOV3 for faster object detection with high speed that uses darknet53 as the base network for their feature extraction. They used Simit Moving datasets for moving object detection and performs the three proposed models. The algorithm used is M-tiny Yolov3.

Limitations:

1. One of the limitations is that it only identifies moving objects as the algorithm used is developed for moving object detection.

2. The other limitation is there is no Distance and Direction factor for their developed paper.

2.2. Object Detection and Recognition using TensorFlow for Blind People

Computer Vision impairment or blindness is one of the biggest disabilities in humans[2.2]. To Overcome the problem of vision impairment. This paper creates a framework to guide the visually impaired on object detection and recognition that leads to performance of independent activities in daily activities

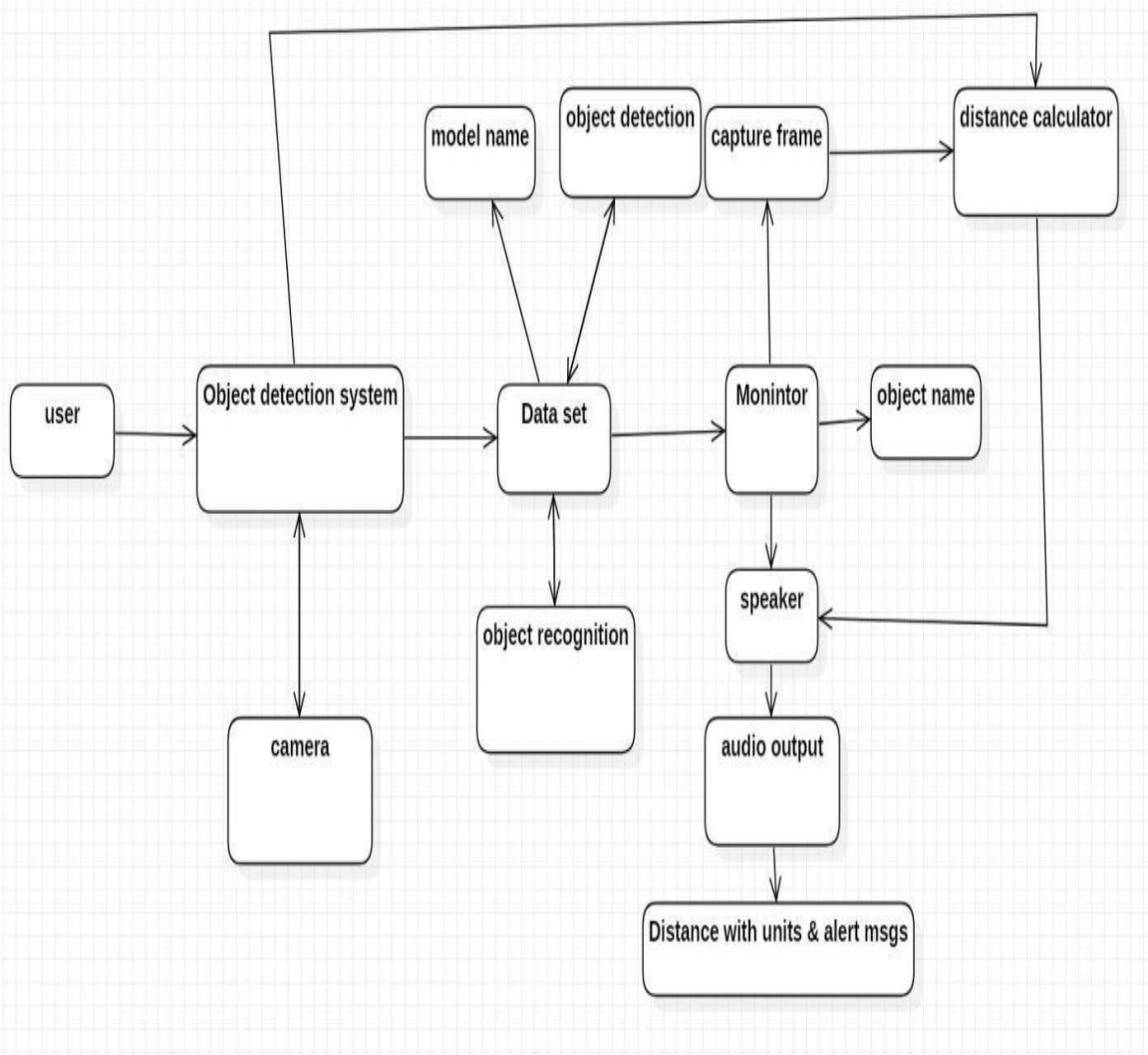


Fig.2.2.System Architecture

- **Video Capturing Module:**

When the system is turned on the system capture images using camera. We have to connect this as input to the COCO dataset and classification of pixels and features takes place. The captured frames can be seen in the monitor with drawn boundaries and label. The method video capture () is used to start the camera and capture the video.

- **Image Processing Module:**

OpenCV (Open-Source Computer Vision) is a library in python which functions mainly aimed at real-time computer vision. It is mainly used to do all the computational operation related to images. cv2 is used to perform image processing and make use of methods which are used to detect and capture the frames and specifies names. This module takes input from the camera.

- **Object Detecting Module:**

The algorithm will take the image as input and all the computations will take place like dividing the image into neurons nothing but pixels and classification of features which will be done on Neural Network. Image will be read as string for the next computation and it will be compared under trained dataset. This can be achieved here by using category index where 90 objects are trained separately. Here we used SSD Architecture which comes under Tensor Flow API.

- **Distance calculation Module:**

The algorithm will take the image as input and all the computations will take place like dividing the image into neurons nothing but pixels and classification of features which will be done neural network. Image will be read as string for the next computation and it will be compared under trained dataset. This can be achieved here by using category index where 90 objects are trained separately. Here we used SSD Architecture which comes under Tensor Flow API.

- **Audio Output Module:**

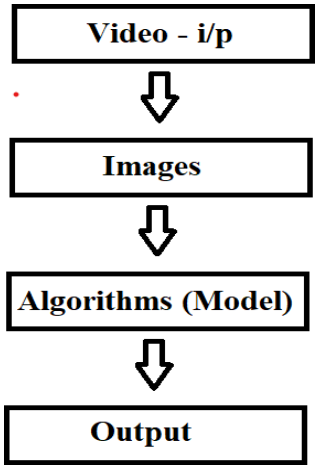
Next after detecting the object and calculating the distance our aim is to give the output in the audio using voice notes. In the output we are going to specify the distance along with units and the warning messages to alert the user. For audio output the pyttxs3 pip package which is predefined python built in module used for converting text to speech.

Limitations:

The framework is only for the system architecture as carrying your system to every place is not a user-friendly object detection to overcome their obstacles in their daily lifestyle.

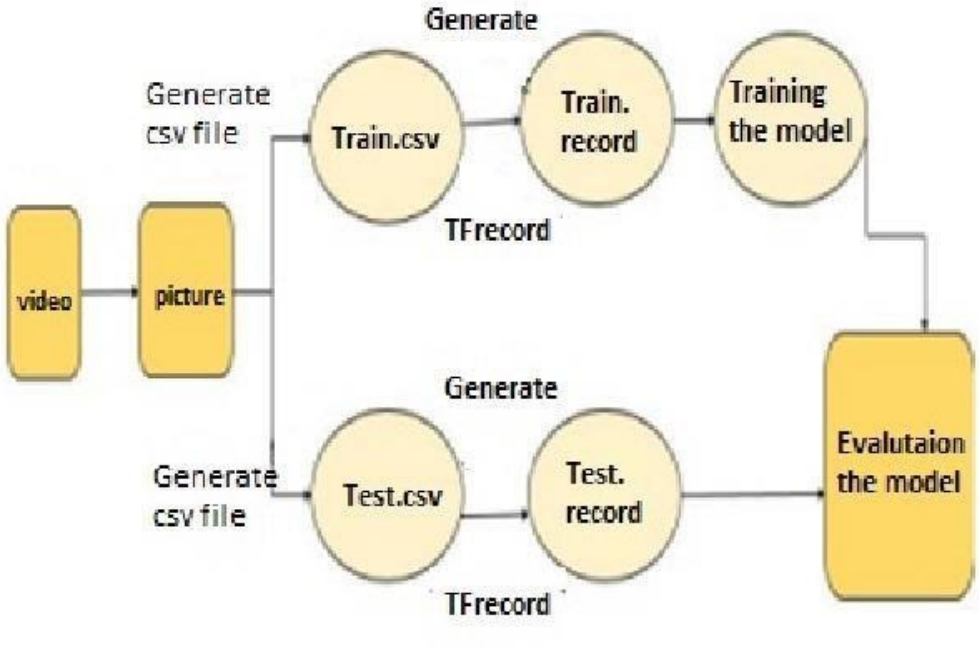
2.3: Detecting objects and people and tracking movements in a video using TensorFlow and deep learning [2020]

The proposed approach permits the analysis of video sequences using deep learning and TensorFlow technologies. This approach splits video in set of images and detects objects that are present in these images and stores their descriptions into a standard XML file[2.3.1]. With an algorithm we developed, we're able track motion that animated entities in the video sequences. The below figures talks about the flow of process:



2.3.1: Process Flow

This approach splits video into a set of images and furthers the process of detection of objects/entities. The information extracted from images is stored in standard XML file. By using R-CNN, color histograms and YOLO, tracking of motion in object from video can be extracted. The ML framework TensorFlow is used for dataflow programming. YOLO – for image classification and object detection[2.3.2]. YOLO is used for image classification, object detection and divides the image into regions & predicts the boundaries and probabilities for each region. The labelling packages are used for detecting objects. If an object/person exists in the field of detection, the XML attributed to each object/person.



2.3.2. Basic flow of Algorithm

1. Motion/feature detection –
    a) Spatial derivatives computation – calculates for each pixel in matrix.
    b) Eigenvalues computation – for each pixel 2 eigenvalues are calculated (> is kept)
    c) Maximum eigenvalue selection
    d) Small eigenvalue removal

e) Corner selection (selected pixel represents points of interests i.e., final points)

2. Optical flow feature tracking – uses Lucas Kanode algorithm for high accuracy, efficiency, robustness.
3. Steps by evaluating workflow:

- Video segmentation
- Annotating image
- Creating tensor flow records
- Training model and Evaluation
- Testing and motion detection

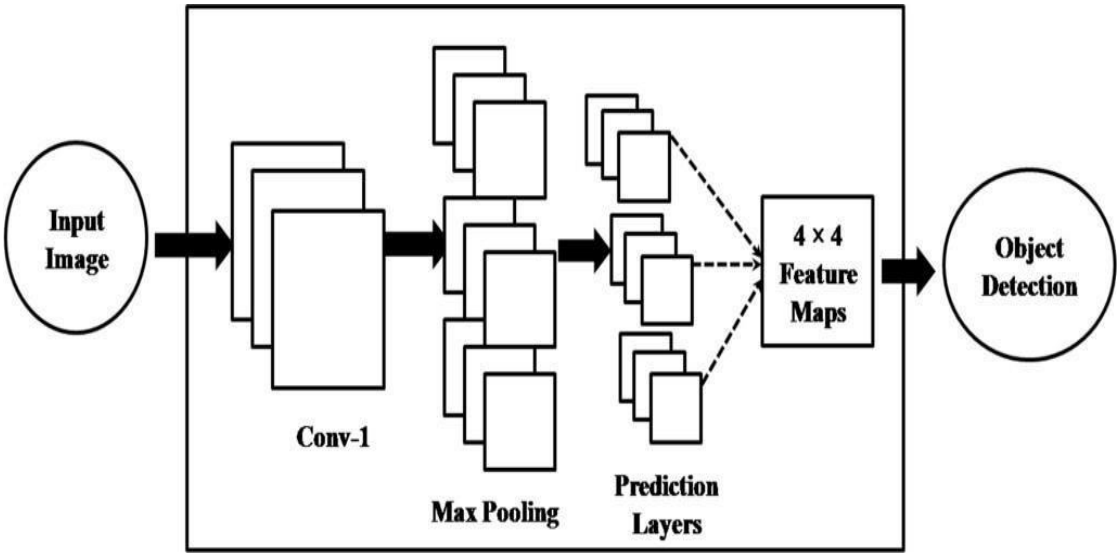
Limitations:

1.It is not deployed into mobile application as discussed by the author earlier.
2.There is no calculation of distance and direction that leads to overcome obstacle.

2.4.An Object Detection Technique for Blind People in Real-Time Using Deep Neural Network

This paper proposed an object detection technique for blind people in real time to detect objects on any device running this model. We use convolutional neural network along with single shot multi-box detector algorithm to develop the proposed model. The single shot multi-box detector algorithm uses standard VOC and COCO datasets [4]. This model takes colour images as input and train the mode until the error rate is less.

The paper presents shorthand for single shot multi-box detector i.e CSSD. In this paper, the authors have used CNN to detect the objects in the live environment. Faster R-CNN with deep Neural Network and single shot detector (SSD) algorithm with additional layers. The SSD algorithm uses feature maps extraction and convolutional filters for detection of small objects.



2.4. Basic flow of algorithm

The training time for this model is about 5-6 hours. This model uses deep neural networks to extract feature information and then perform feature mapping, the SSD Mobile Net FPN performs localization and classification of neural networks and hence is fast compared to other models. In this work, we have used a single shot multi-box detector (SSD) algorithm[2.4] to achieve high accuracy and IOU in real time for detection of the objects for the blind person. A convolutional neural network is used to extract feature information from the image and then perform feature mapping to classify the class label.

1) **Used Technologies:**

Open CV
Pascal VOC and
Coco datasets
F-RCNN
SSD

Limitations:

1.There is no sign of real time video, the detection is only for images.

3. SYSTEM ANALYSIS

3.1. Existing System

In Existing System, visually impaired people face problems to get their daily routines done. They will take the help of people stick, people which are dependent on someone/something. Though there are some ways to help them like smart caps, and blind sticks. But, they are fixed with hardware where if there was any problem they can't resolve it, which makes them feel difficult in such situations.

3.2. Proposed System

Real-time Object detection is widely used for detecting objects from the input image/video given. Detecting various objects from different objects from the images/video along with all the details of the object uses techniques like drawing bounding boxes and providing the probability of the object with certain computations.

The proposed system's goal is to detect objects from the given input video and detected objects and their distance, and direction of the object. The detected are converted into the text to speech which makes users feel comfortable with it and can be completely accessed through mobile phones without any external things like sensors, or Arduino. This project is related to software that only needs mobile for capturing videos and giving voice output.

3.3. Feasibility Study

A feasibility Study is an assessment of the practicality of a proposed plan or method. For feasibility analysis, some understanding of the major requirements for the system is essential.

3.3.1. Technical Feasibility

This study is carried out to check the technical feasibility, that is the technical requirements for this project. The project deals with detecting objects using deep learning techniques. It uses Jupyter Notebook IDE for Python programming, and libraries such as TensorFlow,

tflite, etc. Android Studio is used for Mobile applications. Using these, the objects are detected and the output is given. So, the project is technically feasible within the limits of current technology.

3.3.2. Operational Feasibility

The proposed application identifies around 80 objects in day-to-day life which helps the blind assist and navigate the distance and direction of objects and persons. Hence, the project is operationally feasible.

3.3.3. Economic Feasibility

The proposed system does not need any external resources and is cost-effective. So, the project is economically feasible.

4. SOFTWARE REQUIREMENT SPECIFICATION

4.1. Functional Requirements

Functional Requirements define what the software should do. The Functional requirement implementation in a system is planned in the system design phase. This Functional requirement
is like a product feature, which developers most implement to enable users to complete their tasks.
Functional Requirements for this project are:

- **Input:** Videos , Images , voice commands.
- **Output:** Description of the object detected (Name, distance, direction of object) through voice output.

4.2. Non-Functional Requirements

Non-Functional Requirements define "What a system should be rather than What a system should do. Non-functional Requirements explain the aspects such as Performance, Portability,
Usability etc.

- **Performance:** High Speed
- **Portability:** Can work in different environments like Windows, macOS, linux
- **Usability:** Usability measures the usability of the software system being developed.

➢ **Software Configuration**

Program Language- python

IDE – Jupyter Notebook

Libraries Used – Numpy, Keras, pydttsx3,cv2, speech recognition, Threading

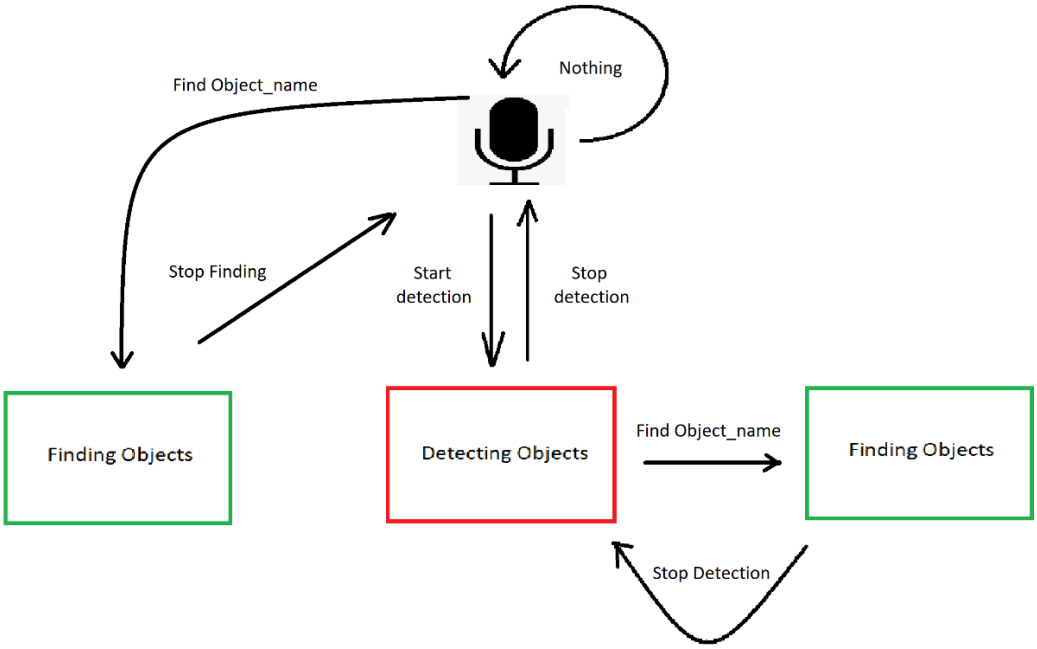
Framework – TensorFlow

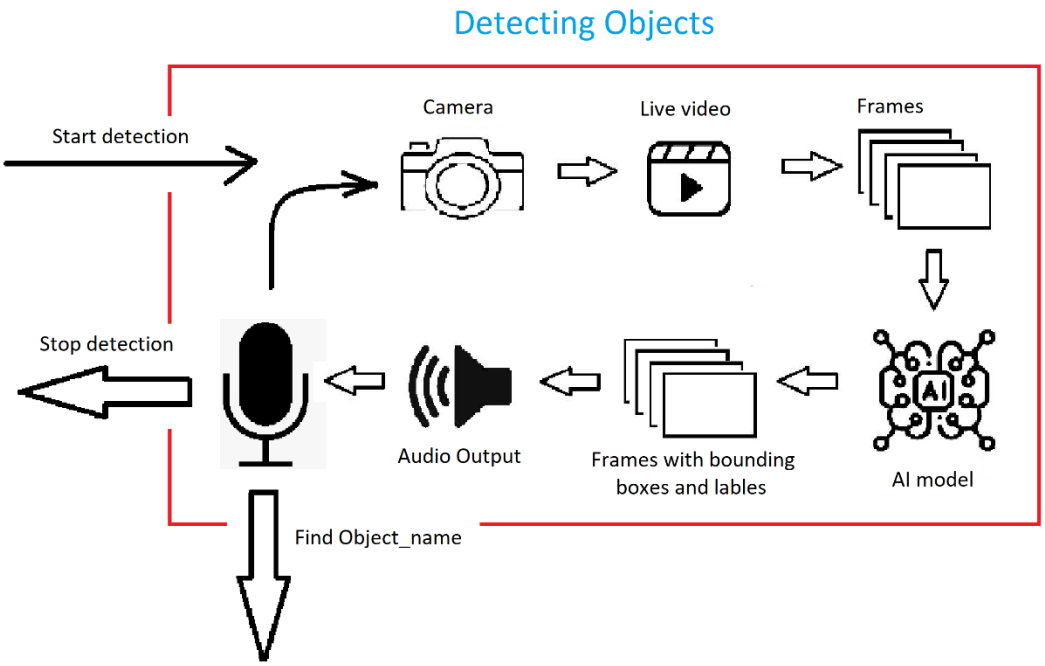
Hardware Configuration:

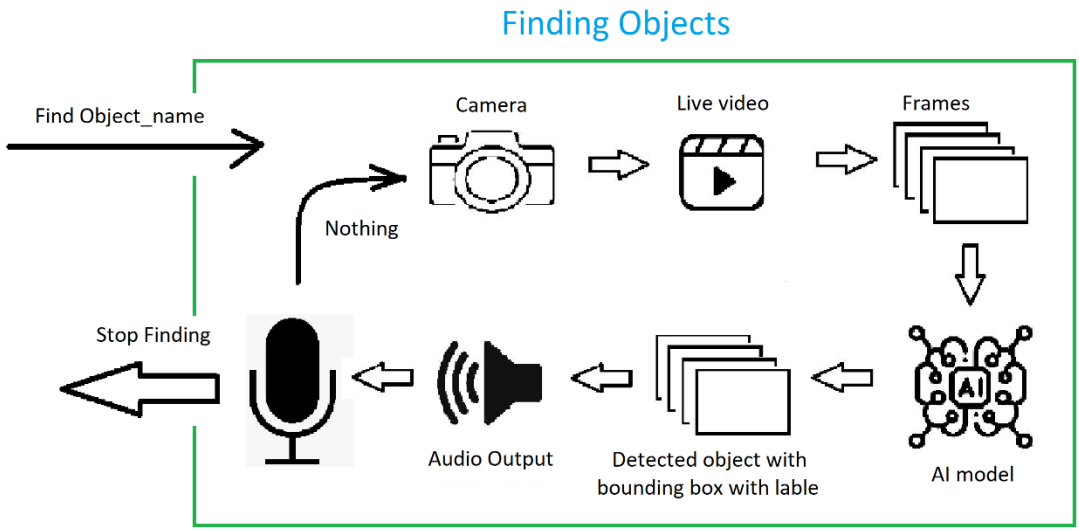
Operating System - Any operating system like windows, macOS, Ubuntu, Linux

5. DESIGN AND METHODOLOGY OF PROPOSED SYSTEM

5.1. System Architecture and Model







5.1. System Architecture

With the help of microphone, the voice command (input) is taken to the inside process of multi-Threading in which the camera takes live video (input) to be captured and the video will be taken as frames. The frames will be given to Artificial intelligence (Deep Learning) Model and by using deep learning techniques. The model will detect the objects in terms of frames with bounding boxes and labels with certain probability [5.1]. The output obtained is converted from text to speech and be given to the user by the particular object if needed.

5.2. System Design

5.2.1.UML Diagrams

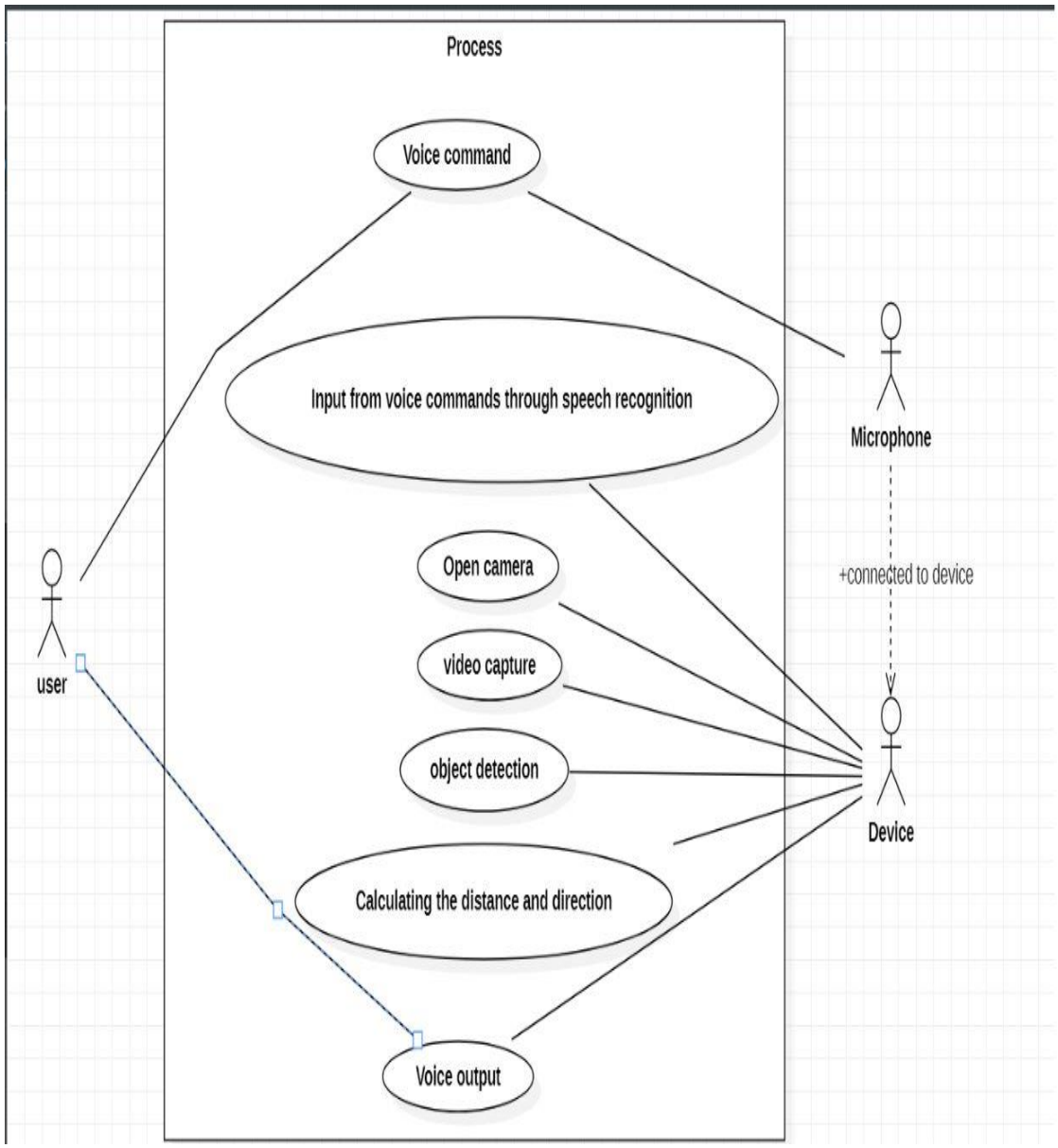
Unified Modelling Language (UML) is a general-purpose modelling language. UML is a way of visualizing a software program using a collection of diagrams. UML is a standard Language for specifying, visualizing, Constructing and Documenting the artifacts of software system. The main aim of UML is define a standard way to visualize are the way a system has been designed. It is quite similar to blueprints used in other fields of engineering. UML is linked with object-oriented design and analysis. UML makes the use of elements and forms associations between them to form diagrams.

Diagrams in UML can be broadly classified as:

1. **Behavioral Diagrams :**
    - Use Case Diagram
    - State Chart Diagram
    - Activity Diagram
    - Sequence Diagram
2. **Structural Diagrams:**
    - Class Diagram
    - Object Diagram
    - Component Diagram
    - Deployment Diagram

A.Use Case Diagram:

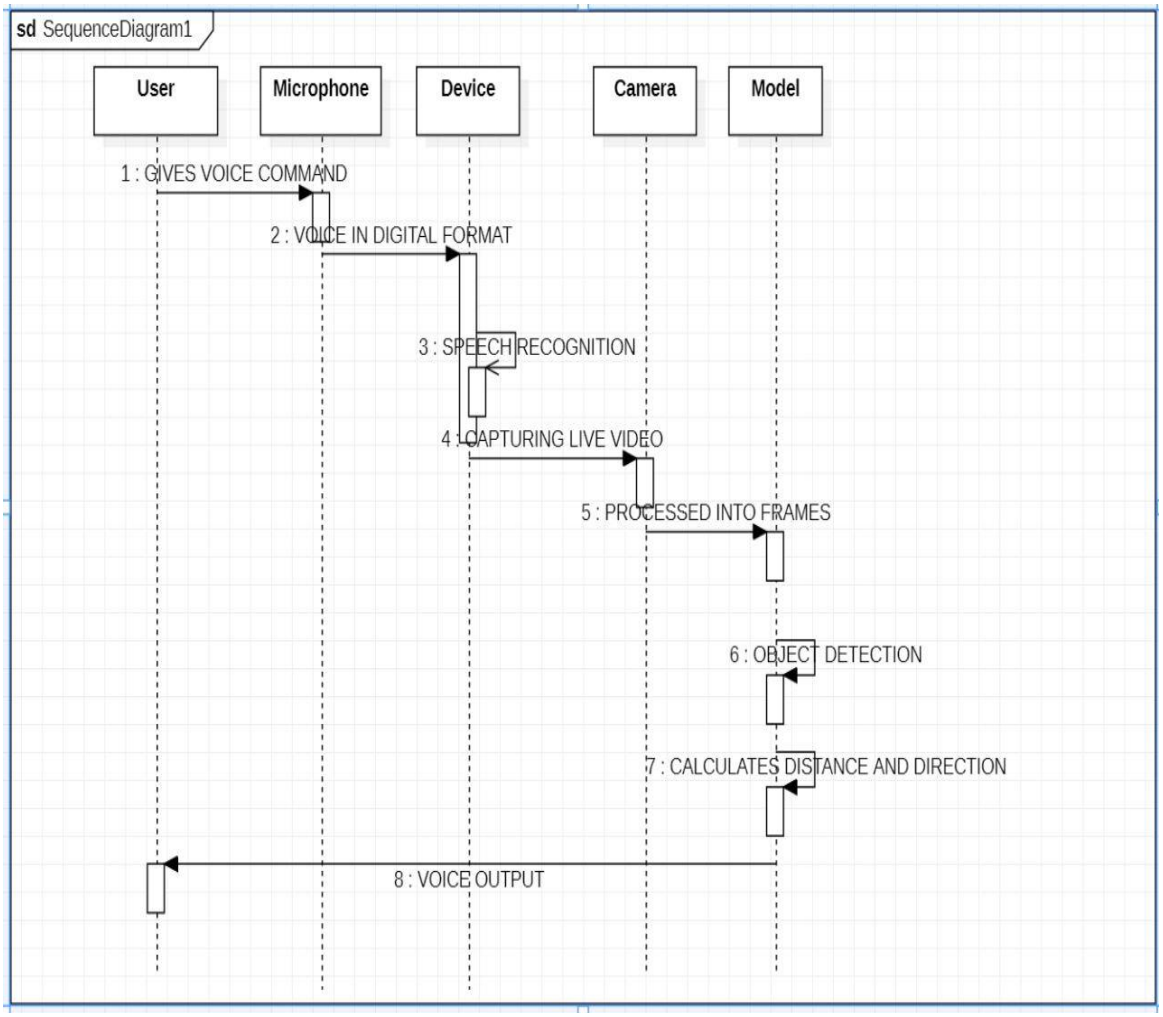
In this model, the use such as voice commands through microphone, opening the camera and voice output were connected with the user and microphone. Video capture, speech recognition, Object detection, direction and distance calculation and UI was connected to the software Application[5.2.1.A]. The actors which are communicated with use cases in this model are the user and the application



5.2.1.A. Use case Diagram

B. Sequence diagram:

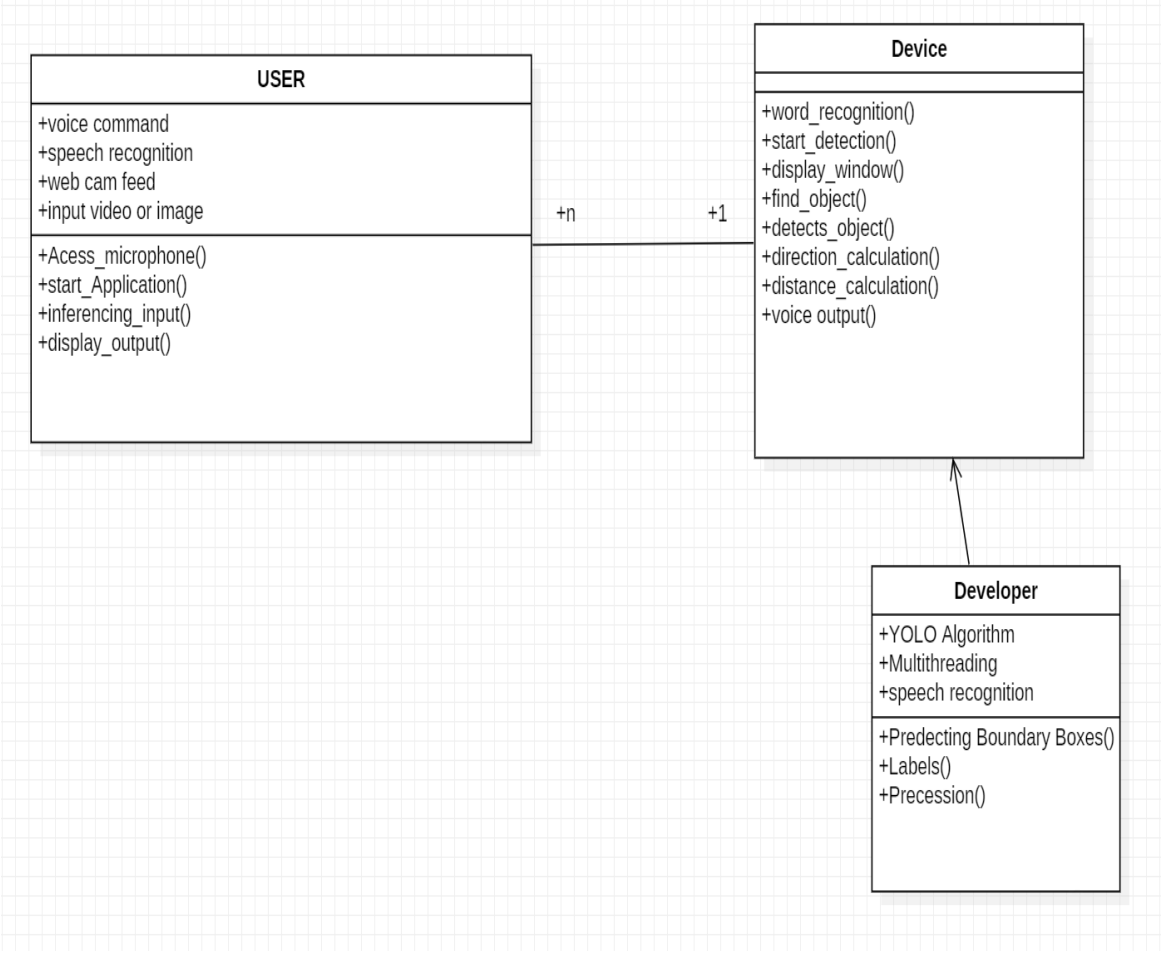
The actors in this model are User, Microphone, Camera, Model, and output. Lifeline user with the help of a voice command from microphone and control flow access camera and then it captures using the camera. Model, with the help of training, detects objects. With the help of the device camera, the model itself calculates the distance and direction which is like a self-call for the model[5.2.1.B]. The user can find the particular object if needed. The output generated is reflected to the user.



5.2.1.B. Sequence Diagram

C. Class Diagram:

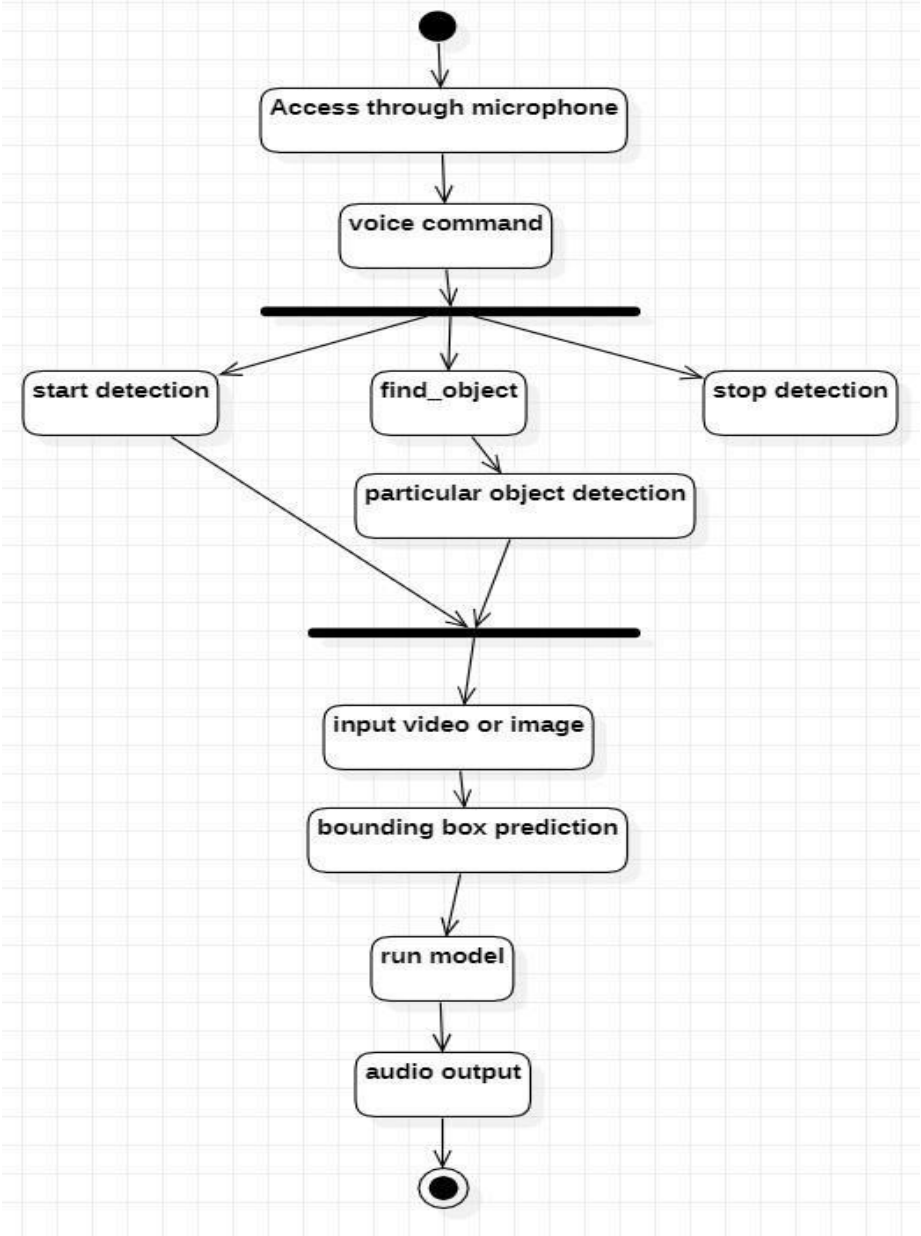
The User class consists of attributes such as input video, voice command, an input image and webcam feed and the operations such as accessing the application, interfacing input, and displaying output. User have a simple association as the relationship to communicate with the application. Application having speech recognition, object detection, displaying windows, calculating distance and direction and voice output as its operations[5.2.1.C]. The developer has attributes such as the YOLO algorithm and operations as predicting bounding boxes, lables, and precision that relates to the application class.



5.2.1.C. Class Diagram

D. Activity Diagram:

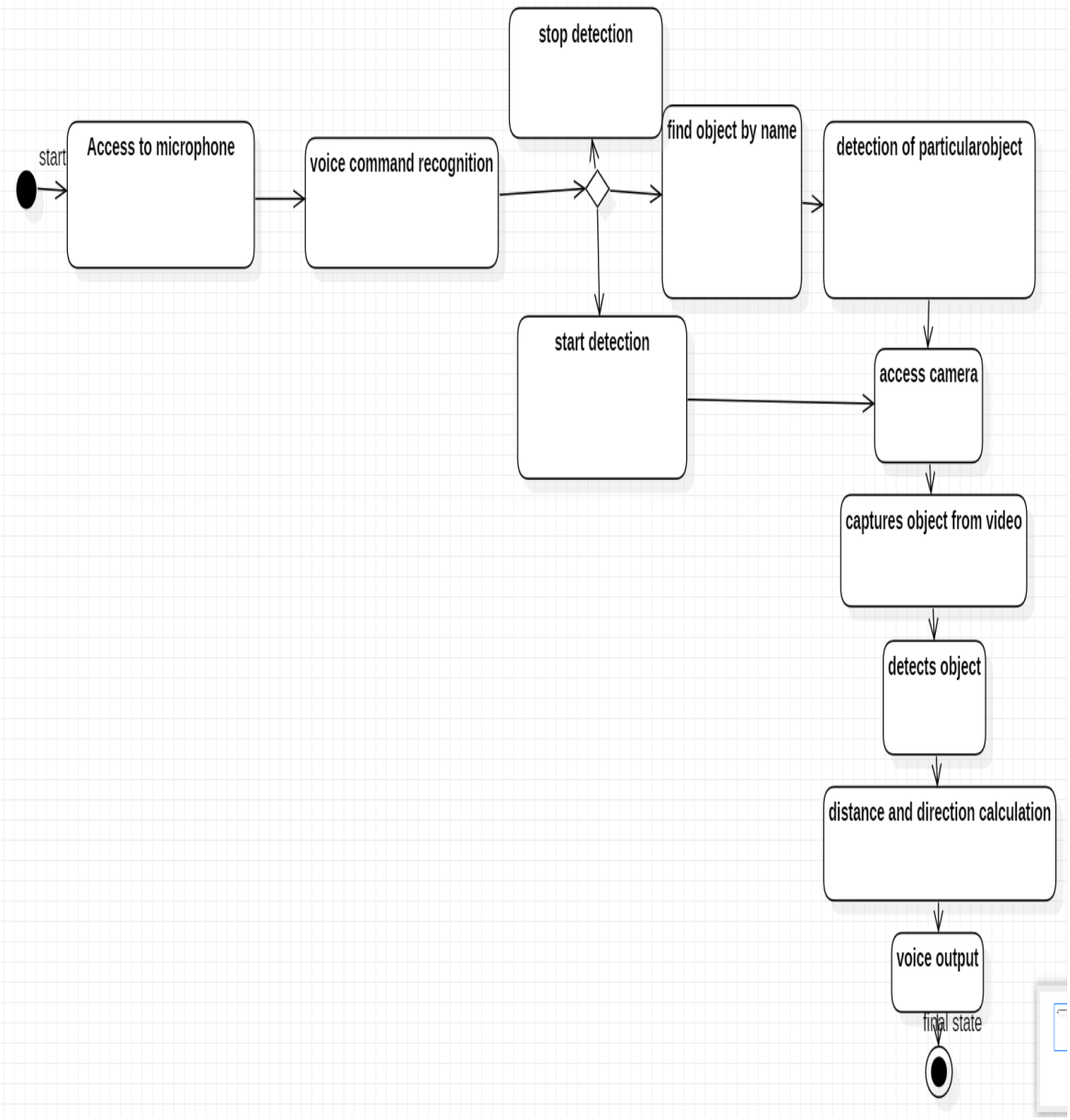
The process of object detection begins by accessing the microphone for voice command the camera is used to take inputs and detecting objects from an image/video with bounding boxes and checking if the objects were present or not with the decision box. If the object is detected, the model calculates distance and direction and gives the audio output through the speaker and reaches to end state[5.2.1.D]. If there is no object to be detected again it access the camera and tries to detect object and the process continues.



5.2.1.D. Activity Diagram

E. State chart Diagram:

In this model, the flow of control moves from one state to another. At the start of the process, the microphone is accessed, and voice commands are taken from it. The cameras are accessed to capture images from video for further processing[5.2.1.E]. From the state of capturing videos, it moves to detect stage and then distance, and direction are calculated and the result is given as voice output via the phone speaker and reaches the end state.



5.2.1.E. State Chart Diagram

5.3. Methodology

5.3.1. Process Model/Methodology/Algorithms

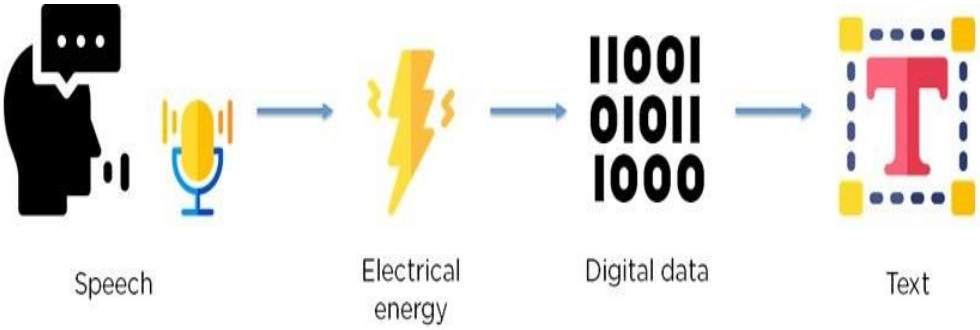
The model takes voice command as input and uses speech recognition to find the particular object needed. Further, the Model is built using YOLOv3 algorithm. YOLOv3 (You Only Look Once, Version 3) is a real-time object detection algorithm that identifies specific objects from taken input and detects image/videos from camera. It is one of the popular algorithms for object detection which uses deep learning. To pass through the network YOLO only needs one forward propagation. YOLO has 9 versions to date. It is getting enhancement in every version of it and increasing its accuracy in predictions.

5.3.1.1. Speech recognition in NLP

- One of the most significant benefits of speech recognition One of the most significant benefits of speech recognition technology is that it can be used as a communication aid for people with disabilities. Blind individuals, in particular, can benefit from this technology because it allows them to access information through spoken commands instead of relying on visual cues.
- Speech recognition technology can help blind individuals communicate more effectively, improve their quality of life and enhance their autonomy. Speech recognition software can assist blind individuals by allowing them to interact with different devices such as computers and smartphones verbally.
- This eliminates the need for keyboards or other input devices that rely on touch screens that require visual confirmation or interpretation.
- With speech recognition technology integrated into these systems, visually impaired users can easily operate electronic devices by issuing voice commands.
- It focuses on retranslation of speech from a verbal format to a text format whereas voice recognition focuses on identifying the individual voice. In Python, speech recognition works with deep neural networks and hidden Markov model algorithm that perform linguistic and acoustic modeling.
- Linguistic modeling is used to perform the study of different languages whereas

acoustic modeling performs to recognize the language based on the sound of the speech it converts into text format that do get more meaningful sentences.

- For speech recognition it uses models such as hidden Markov model to understand the very complex human language. Human language isn't so simple as as people speak with different accents, slang and mispronunciation.



5.3.1.1. speech recognition

- Speech recognition begins with audio acquisition which an audio signal is captured using a microphone
- The audio signal is preprocessed what to remove any noise that might make the recognition process difficult voice will be breaking down into individual sounds and converting this sounds into digital format [5.3.1.1]. The audio signal is then transformed into a feature extraction, where the analogue to digital signal is converted using a microphone.
- The next step is to train the acoustic model that recognizes pina. This is done by using Michael learning algorithms that identify patterns in the acoustic features and associate them with specific phonemes.
- Hidden Markov model is used for this speech recognition. Once the model is trained, a language model is developed to recognize words. This model is used to find the temporal patterns in speech and improve accuracy.
- The three voice commands that the model recognizes are:
    - ❖ Start detection(): This command will enable the model to start a thread and process detection of all objects.
    - ❖ find by object name(): This command will enable the model to find the specific object mentioned by the user.

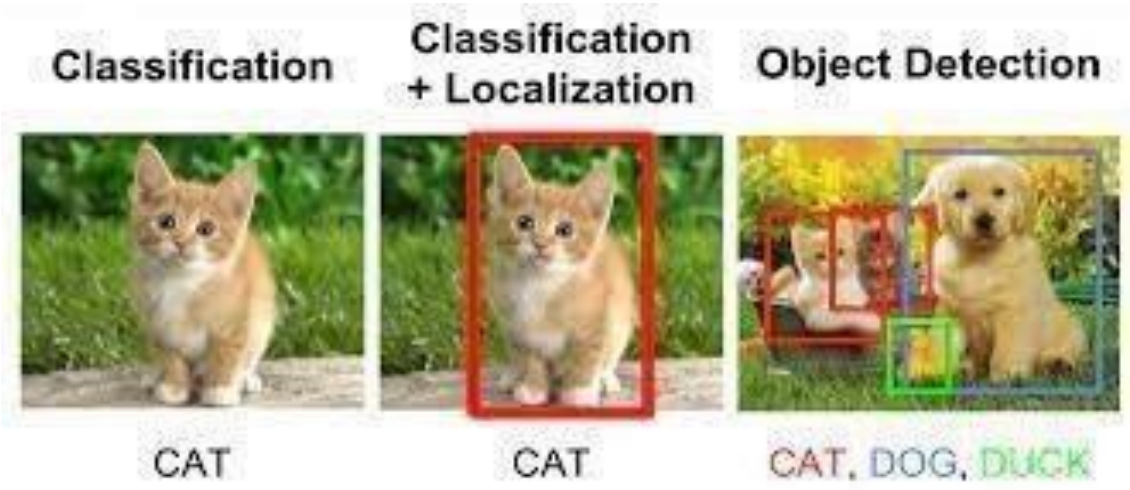
- ❖ Stop detection():The user can stop the detection process when needed by using the “stop Command”.

- Finally, the system uses acoustic and language models to decode the voice and generates transcription of spoken words. Overall, when we give any command, it converts to text format

5.3.1.2. Classification vs Localization vs Detection

Image Classification is nothing but classifying the image. For example, an image contains a c, at and image classification is done by classifying it as a cat.

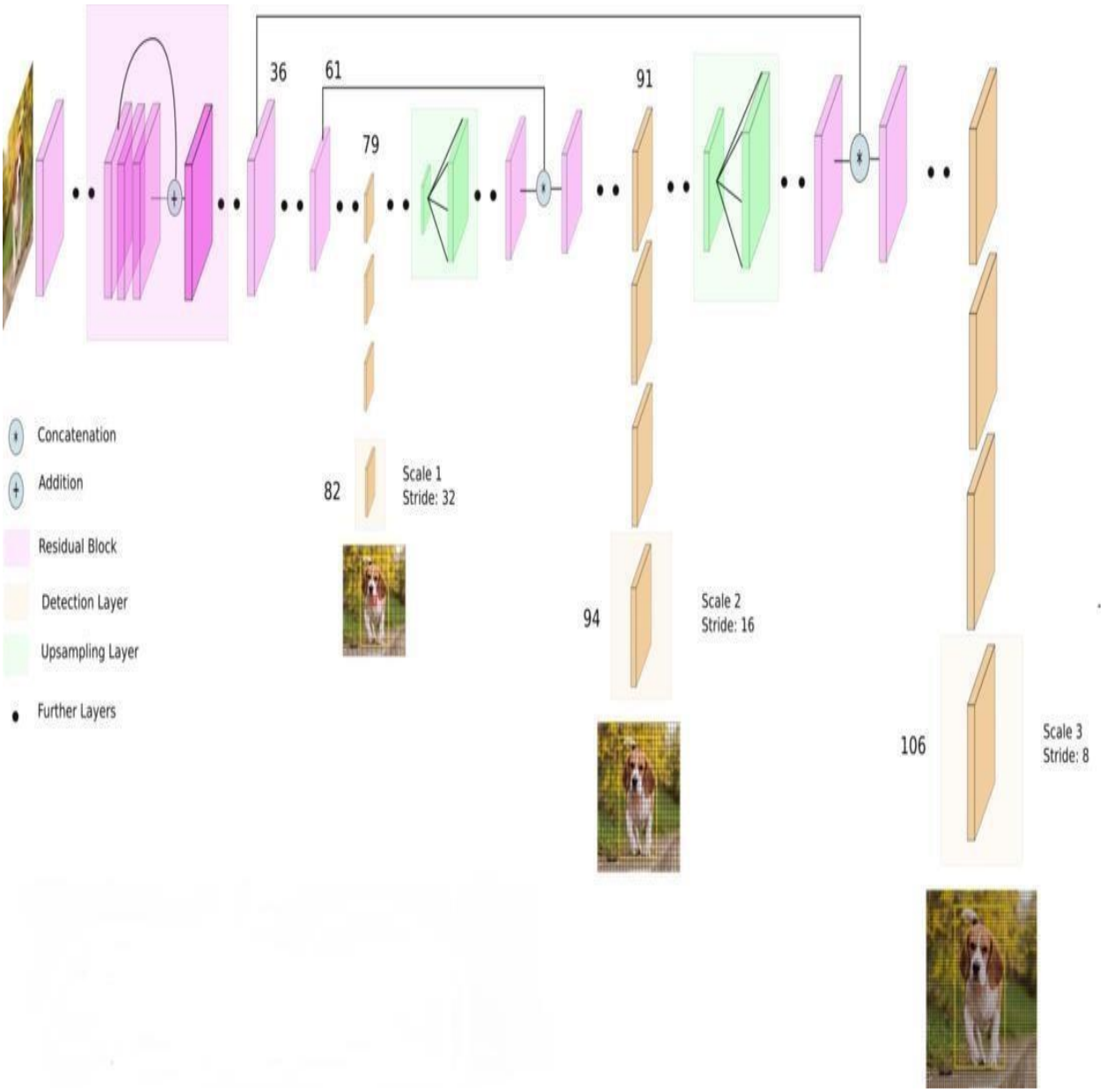
- Localization will specify the location of a single object in an image whereas Object Detection specifies the location of multiple objects in the image. For example, an image contains a cat, and the cat will be specified with a bounding box, and Localization will specify the location of a single object in an image.
- Object detection is a technique that detects the local instances in an image/video. For example, an image contains a cat, a dog, and a duck [5.3.1.2]. Object detection can find multiple objects in an image i.e, in this example, it can detect all the cats, dogs, and ducks.



5.3.1.2. Classification vs Localization vs Detection

5.3.1.3.Architecture of YOLO V3:

YOLO v3 uses a variant of Darknet, which originally has 53 layer network trained on Image net. For the task of detection, 53 more layers are stacked onto it, giving us a 106 layer fully convolutional underlying architecture for YOLO v3.



5.3.1.3.1. Layers in YOLO v3

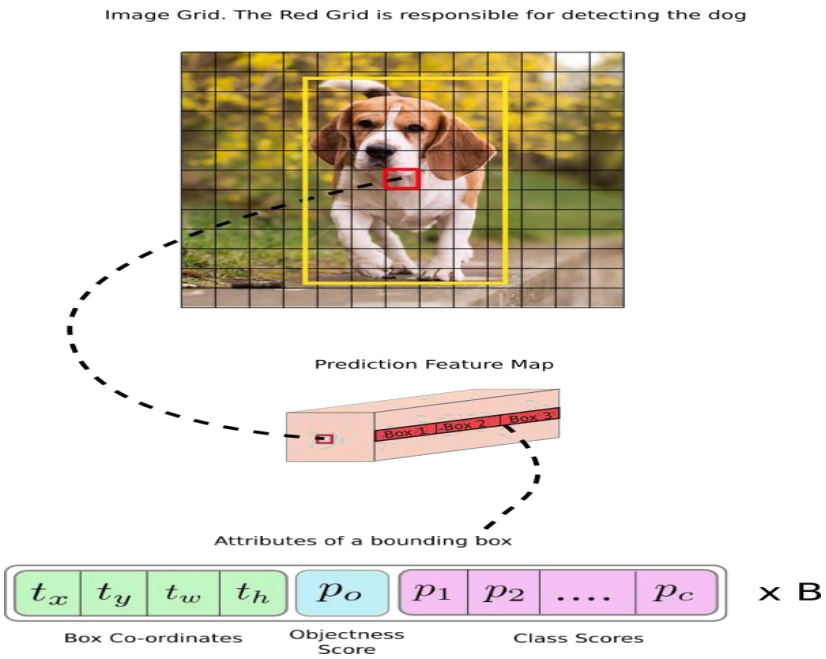
YOLO v3 detects objects at three different scales which is noted to be the salient feature of this algorithm[5.3.1.3.1]. The detection is done by applying 1 x 1 detection kernels on feature maps of three different sizes at three different places in the network.

The shape of the detection kernel is 1 x 1 x (B x (5 + C)), where B is Number of Anchor Boxes in a cell predicted in feature map,"5" is divided as (4+1) where 4 are the values of Bounding Box coordinates and 1 is object confidence,C is the class scores (The number of class scores depends on the no of objects trained i.e. COCO dataset Can predict 80 objects)

Note: stride of the network, or a layer is defined as the ratio by which it down samples the input.

YOLO v3 makes prediction at three scales, which are precisely given by down sampling the dimensions of the input image by 32, 16 and 8 respectively. The input image is taken in size 416x416. This image is down sampled till the 81st layer, such that 81st layer have a stride of 32. Then the first detection is done in the 82nd layer by applying 1x1 kernel on the 13x13 down sampled image. The out produced will be a detection feature map of 13 x 13 x 255.

Refer the diagram for a clear view.



5.3.1.3.2. Detecting objects at three scales

As we see the image is downsampled to a 13x13 grid. Now for each cell in the grid three bounding boxes will be drawn. For each Bounding box, the out will be 85 values(coordinates[4] + objectness score[1] + class scores[80], refering to the COCO dataset). So for each cell, the output will be Bx85, where B is the number of Anchor boxes.

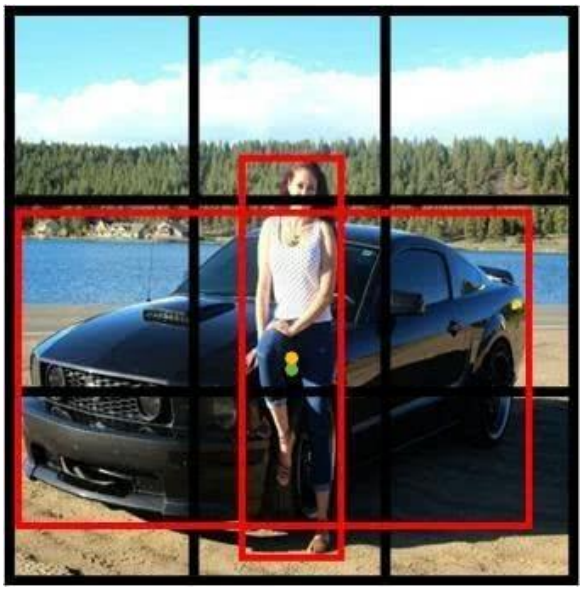
Then, the feature map from layer 79 is subjected to a few convolutional layers before being up sampled by 2x to dimensions of 26 x 26. This feature map is then depth concatenated with the feature map from layer 61[5.3.1.3.2]. Then the combined feature maps is again subjected a few 1 x 1 convolutional layers to fuse the features from the earlier layer (61). Then, the second detection is made by the 94th layer, yielding a detection feature map of 26 x 26 x 255.

A similar procedure is followed again, where the feature map from layer 91 is subjected to few convolutional layers before being depth concatenated with a feature map from layer 36. Like before, a few 1 x 1 convolutional layers follow to fuse the information from the previous layer (36). We make the final of the 3 at 106th layer, yielding feature map of size 52 x 52 x 255.

5.3.1.4. Anchor Boxes:

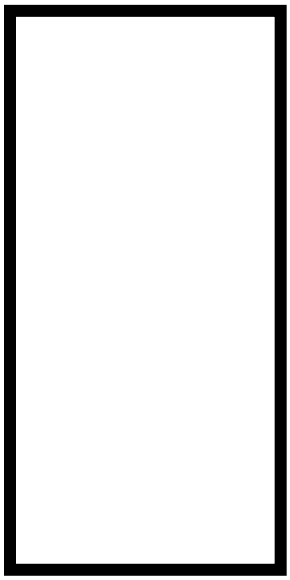
In detecting objects in a cell, generally a cell returns only one object in the cell. But what if there are 2 objects in the cell[5.3.1.4]. So the concept of anchor boxes are used. Using multiple anchor boxes and making prediction on them gives us better results. It solves the problem of detecting multiple objects in a cell.

As you can see in the image below two objects in the same cell can be detected using anchor boxes.



5.3.1.4.1Anchor Boxes

Here,2 anchor boxes are used



Anchor box 1

Anchor box 2

The values of anchor boxes are as follows:

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Here the output had one anchor box. If the output have 2 anchor boxes the output will be as shown in the example below:

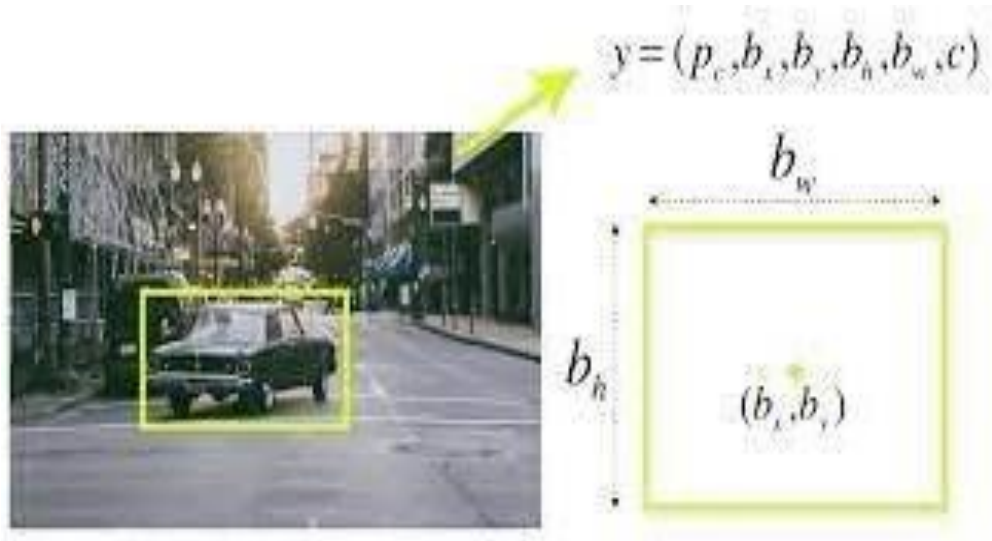
$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 1 \\ 0 \\ 0 \\ 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix} \begin{matrix} \text{Anchor box 1} \\ \\ \text{Anchor box 2} \end{matrix}$$

Yolo v3 uses 9 set of anchors boxes. i.e. 3 set of anchor boxes at each detection level. If you're training YOLO on your own dataset, you should go about using K-Means clustering to generate 9 anchors.

5.3.1.5. Predicting Bounding Boxes:

A Bounding Box is a outline that highlights the object in a image. Every Bounding Box Consists of these attributes:

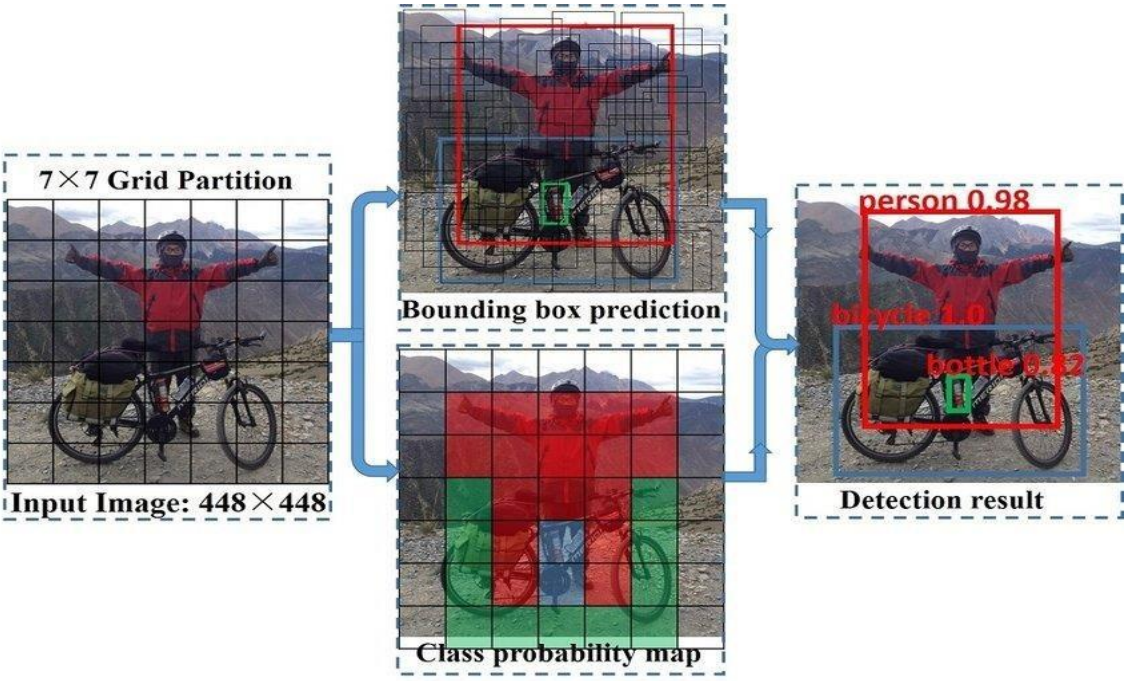
- Width
- Height
- Class (for ex: cat, dog)
- Bounding Box center



5.3.1.5.1. Predicting Bounding boxes

Here the bounding box is drawn to the car [5.3.1.5.1]. The attribute of the Bounding box are shown in the right of the image where:

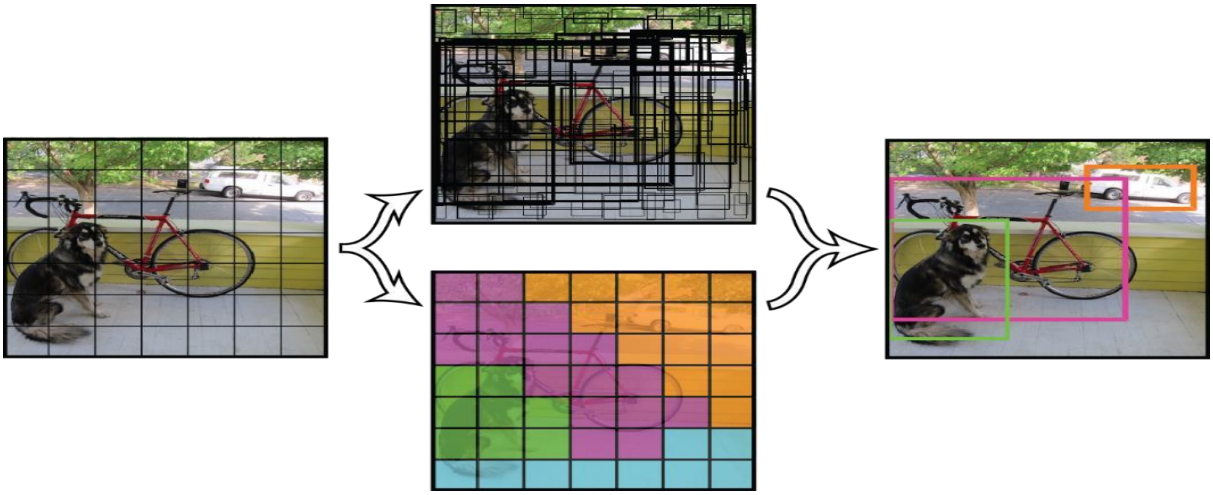
Pc is object ness score
bx , by are the coordinates
bh , bw are the
height and width c is
the class i.e. car



5.3.1.5.2. Predicting Bounding boxes in Steps

The above image is a clear example of predicting the Bounding Box from the image. Mapping is performed over here to combine all the similar classes and produce a Bounding Box[5.3.1.5.2]. Sigmoid function is applied on the anchor box outputs except the height and width.

Examples of Bounding Boxes are:

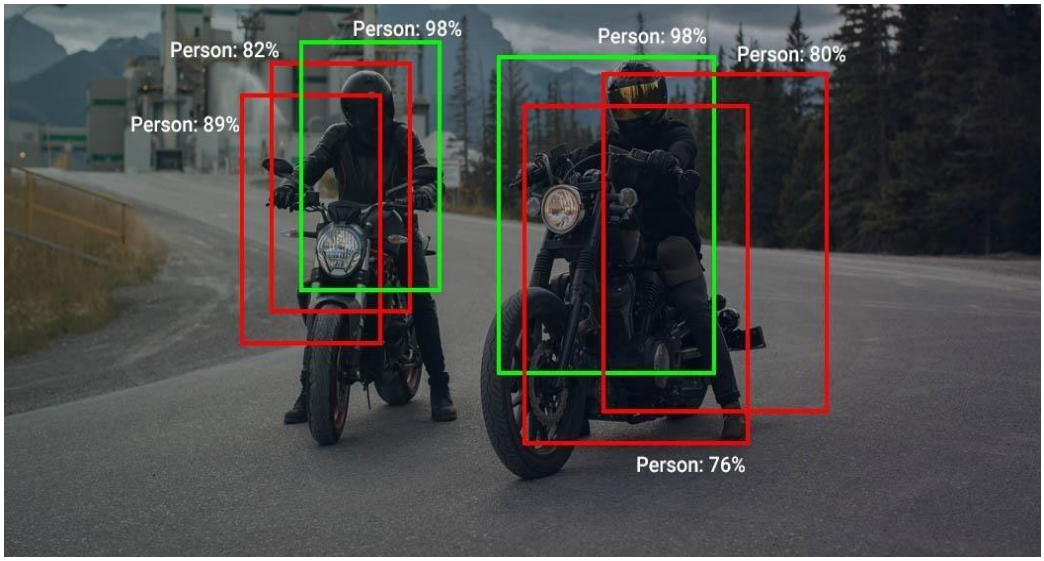


5.3.1.5.3. Predicting Bounding boxes in Images

So as of now, an object gets an single Bounding Box at the end of detection. As YOLO v3 does detection at three different levels, we use non max suppression to get the current correct bounding box from the three boxes.

5.3.1.6. Non-Max Suppression:

Currently, YOLOv3 uses a Non-Max Suppression algorithm to eliminate multiple detections of the same object[5.3.1.6]. This algorithm is picking the most confident overlaying box on any object to present it as the final prediction.



5.3.1.6. Non-Max Suppression

5.3.1.7 Distance Calculation module

Motion analysis and trajectory-based tracking are the major techniques used to find the object's location in the frame of the video. Additionally, determining the distance from a camera to an object is a very well-studied problem in computer vision and image processing space. The triangular similarity is the major technique used for distance calculation. for example, a known object with width W placed at some distance D from the camera. We take a picture of our object using a camera and then measure the apparent width in pixels P. The focal length is derived as:

$F = (P \times D) / W$

The continuation of moving the camera closer and farther away from the object can use triangle similarity to determine the distance of the object to the camera as:

$D' = (W \times F) / P$

The first step to finding the distance to an object in an image is to calibrate and compute the focal length. To perform this, the main two rules are:

1. The camera's distance from an object
2. The width/height of the object

we can compute the distance from our camera to our object in images/videos using the calibration of our system(measured in terms of pixels per metric) and the focal length. Better accuracy can be seen by computing the extrinsic and intrinsic parameters.The most significant way is to perform a checkerboard camera calibration using OpenCV. This will remove radial distortion and tangential distortion, both of which impact the output image, and therefore the output measurement of objects in the image.

5.3.1.8.Direction calculation:

The direction calculation of the detected object can be measured by using the ratio 1:2:1.these ratios are divided according to the screen size (416x416) to determine wheather the object is placed in left or right or center near the person.

5.3.2. Tools & Techniques

- **Jupyter Notebook IDE** is used to build and run the model. Jupyter Notebook has a flexible interface, which allows users to configure and arrange workflows in data science, scientific computing, computational journalism, and machine learning. The entire model will be built and run here.

6.MODULES

6.1. IMPLEMENTATION OF BASE PAPER

6.1.1. Speech Recognition

Objects will detected the speech from the microphone accessed and process it to the model for object detection.

6.1.2. Object Detection

Objects will be detected from the voice input (find object()).Objects will be detected continuously from the image/video if there is no voice command .

6.1.3. Direction

Direction is calculated to specify the user in which direction does other objects/ things have.

6.1.4 Distance

Distance is calculated by using focal length to let the use know the distance from the distance from the user to other objects/persons.

6.1.5. Voice Output

The calculated distance, direction and the objects detected will be an text as an resultant Output/ that text is converted into an speech output.

6.2. Description of sample code of Each Module

6.2.1. Making of Model with yolo v3 weights and saving the model using keras:

```
import struct  import numpy as np
from keras.layers import Conv2D
from keras.layers import Input
from keras.layers import BatchNormalization
from keras.layers import LeakyReLU
from keras.layers import ZeroPadding2D from keras.layers
import UpSampling2D from keras.layers
```

```
import add, concatenate from keras.models import Model
#this function takes the input of layers configuration and #create layers with residual
connection or shortcut(if needed) def _conv_block(inp, convs, skip=True):
x = inp
  count = 0
for conv in convs:
if count == (len(convs) - 2) and skip:
skip_connection = x count += 1
if conv['stride'] > 1: x = ZeroPadding2D(((1,0),(1,0)))(x)
 #padding as darknet prefer left and top
x = Conv2D(conv['filter], conv['kernel'], strides=conv['stride'],padding='valid' if
conv['stride'] > 1 else 'same',
# padding as darknet prefer left and top
name='conv_' + str(conv['layer_idx']),
use_bias=False if conv['bnorm'] else True)(x)
If conv['bnorm']:
x=BatchNormalization(epsilon=0.001,name='bnorm_'+str(conv['layer_idx']))(x)
if conv['leaky']: x = LeakyReLU(alpha=0.1, name='leaky_' + str(conv['layer_idx']))(x)
return add([skip_connection, x]) if skip else x
#106 layers of yolo v3 are defined here def make_yolov3_model():
input_image = Input(shape=(None, None, 3))
# Layer 0 => 4
x = _conv_block(input_image, [{'filter': 32, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky':
True, 'layer_idx': 0},
{'filter': 64, 'kernel': 3, 'stride': 2, 'bnorm': True, 'leaky': True, 'layer_idx': 1},
{'filter': 32, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 2},
{'filter': 64, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 3}])
# Layer 5 => 8
x = _conv_block(x, [{'filter': 128, 'kernel': 3, 'stride': 2, 'bnorm': True, 'leaky': True,
'layer_idx': 5},
{'filter': 64, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 6},
```

```
{'filter': 128, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 7}])
# Layer 9 => 11
x = _conv_block(x, [{'filter': 64, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True,
'layer_idx': 9},
{'filter': 128, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 10}])
# Layer 12 => 15
x = _conv_block(x, [{'filter': 256, 'kernel': 3, 'stride': 2, 'bnorm': True, 'leaky': True,
'layer_idx': 12},
{'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 13},
{'filter': 256, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 14}])
# Layer 16 => 36 for i in range(7):
x = _conv_block(x, [{'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True,
'layer_idx': 16+i*3},
{'filter': 256, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 17+i*3}])
skip_36 = x
# Layer 37 => 40
x = _conv_block(x, [{'filter': 512, 'kernel': 3, 'stride': 2, 'bnorm': True, 'leaky': True,
'layer_idx': 37},
{'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 38},
{'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 39}])

# Layer 41 => 61 for i in range(7):
x = _conv_block(x, [{'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True,
'layer_idx': 41+i*3},
{'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 42+i*3}])
skip_61 = x

# Layer 62 => 65
x = _conv_block(x, [{'filter': 1024, 'kernel': 3, 'stride': 2, 'bnorm': True, 'leaky': True,
'layer_idx': 62},
{'filter': 512, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 63},
```

```
{'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 64}])

# Layer 66 => 74
for i in range(3):
x = _conv_block(x, [{'filter': 512, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True,
'layer_idx': 66+i*3},
{'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 67+i*3}]) #
Layer 75 => 79
x = _conv_block(x, [{'filter': 512, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True,
'layer_idx': 75},
{'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 76},
{'filter': 512, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 77},
{'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 78},
{'filter': 512, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 79}],
skip=False)

# Layer 80 => 82 # ----- shortcut------->
yolo_82 = _conv_block(x, [{'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky':
True, 'layer_idx': 80},
{'filter': 255, 'kernel': 1, 'stride': 1, 'bnorm': False, 'leaky': False, 'layer_idx': 81}],
skip=False)

# Layer 83 => 86                                    >
x = _conv_block(x, [{'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True,
'layer_idx': 84}], skip=False)
x = UpSampling2D(2)(x)
x = concatenate([x, skip_61])

# Layer 87 => 91
x = _conv_block(x, [{'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True,
'layer_idx': 87},
```

```
{'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 88},
{'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 89},
{'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 90},
{'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 91}],
skip=False)

# Layer 92 => 94
yolo_94 = _conv_block(x, [{'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True,
'layer_idx': 92},
{'filter': 255, 'kernel': 1, 'stride': 1, 'bnorm': False, 'leaky': False, 'layer_idx': 93}],
skip=False)
# Layer 95 => 98
x = _conv_block(x, [{'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True,
'layer_idx': 96}], skip=False)
x = UpSampling2D(2)(x)
x = concatenate([x, skip_36])

# Layer 99 => 106
yolo_106 = _conv_block(x, [{'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky':
True, 'layer_idx': 99},
{'filter': 256, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 100},
{'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 101},
{'filter': 256, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 102},
{'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 103},
{'filter': 256, 'kernel': 3, 'stride': 1, 'bnorm': True, 'leaky': True, 'layer_idx': 104},
{'filter': 255, 'kernel': 1, 'stride': 1, 'bnorm': False, 'leaky': False, 'layer_idx': 105}],
skip=False) model = Model(input_image, [yolo_82, yolo_94, yolo_106])
return model
#this class loads the weights to the model class WeightReader:
def     init (self, weight_file):
with open(weight_file, 'rb') as w_f:
```

```
major, = struct.unpack('i', w_f.read(4)) minor, = struct.unpack('i', w_f.read(4)) revision, =
struct.unpack('i', w_f.read(4))
if (major*10 + minor) >= 2 and major < 1000 and minor < 1000: w_f.read(8)
else:
w_f.read(4)
transpose = (major > 1000) or (minor > 1000) binary = w_f.read()
self.offset = 0
self.all_weights = np.frombuffer(binary, dtype='float32')

def read_bytes(self, size):
self.offset = self.offset + size
return self.all_weights[self.offset-size:self.offset]
def load_weights(self, model):
 for i in range(106):
try:
conv_layer = model.get_layer('conv_' + str(i)) print("loading weights of convolution #" +
str(i)) if i not in [81, 93, 105]:
norm_layer = model.get_layer('bnorm_' + str(i)) size =
np.prod(norm_layer.get_weights()[0].shape) beta = self.read_bytes(size) # bias
gamma = self.read_bytes(size) # scale
 mean = self.read_bytes(size) # mean
 var = self.read_bytes(size) # variance
weights = norm_layer.set_weights([gamma, beta, mean, var])
 if len(conv_layer.get_weights()) > 1:
bias   = self.read_bytes(np.prod(conv_layer.get_weights()[1].shape))
 kernel = self.read_bytes(np.prod(conv_layer.get_weights()[0].shape))
 kernel = kernel.reshape(list(reversed(conv_layer.get_weights()[0].shape)))
 kernel = kernel.transpose([2,3,1,0])
conv_layer.set_weights([kernel, bias]) else:
kernel = self.read_bytes(np.prod(conv_layer.get_weights()[0].shape))
 kernel = kernel.reshape(list(reversed(conv_layer.get_weights()[0].shape)))
```

```
 kernel = kernel.transpose([2,3,1,0])
conv_layer.set_weights([kernel]) except ValueError:
print("no convolution #" + str(i))
def reset(self):
self.offset = 0

# define the model
model = make_yolov3_model()

#download the yolov3 weights from the official website
https://pjreddie.com/darknet/yolo/ and pass the file as input.
weight_reader = WeightReader('yolov3-416.weights')

weight_reader.load_weights(model)
```

```
loading weights of convolution #0
loading weights of convolution #1
loading weights of convolution #2
loading weights of convolution #3
no convolution #4
loading weights of convolution #5
loading weights of convolution #6
loading weights of convolution #7
no convolution #8
loading weights of convolution #9
loading weights of convolution #10
no convolution #11
loading weights of convolution #12
loading weights of convolution #13
loading weights of convolution #14
no convolution #15
loading weights of convolution #16
loading weights of convolution #17
no convolution #18
```

6.2.1.1.Weights of the Model

```
#saving the model model.save('model.h5')
```

The model will be saved in the same directory as follows:

6.2.1.2.saved model.h5

6.2.2. Speech recognition

6.2.2.1. Main Function

```
def main():
   while(1):
      print('Entered main function .....')
      try:
            with sr.Microphone() as source2:
             r.adjust_for_ambient_noise(source2, duration=0.2)
             audio2 = r.listen(source2)
             MyText = r.recognize_google(audio2)
             MyText = MyText.lower()
             names = list(MyText.split())
             print(names)
             print("Did you say ",MyText)
             if 'start' in MyText:
                speak('detecting')
                condition = start_detection()
                if condition:
```

```
                start_detection()
            elif 'find' in MyText:
                found = False
                obj_name = 'none'
                for i in names:
                    if i in labels:
                        found = True
                        obj_name = i
                        break
                if obj_name == 'none':
                    speak('cant detect object')
                else:
                    temp = 'finding '+obj_name
                    speak(temp)
                    start_detection(True,obj_name)
            elif 'stop' in MyText:
                break
        except sr.RequestError as e:
            print("Could not request results; {0}".format(e))
        except sr.UnknownValueError:
            print("unknown error occurred")
main()
```

6.2.2.2. Start Detection Function

```
def start_detection(finding = False,obj_name = 'none',start_find = False):
video = cv2.VideoCapture(0)
# video = cv2.VideoCapture('sample.mp4')
count = 0
condition = False
global cmd,flag
flag = True
cmd = []
if finding == False:
    while True:
        ret , frame = video.read()
        if ret == True:
            count += 1
            size = (416,416)
            frame = cv2.resize(frame,size)
            if count % 48 == 0:
                name = 'frame {} in {} sec.png'.format((count//5),count//32)
                cv2.imwrite(name,frame)
                print('Entered start_detection function .....')
#                   image_detection('testing.png')
                    image_detection(name)
                t = threading.Thread(target=mic)
                t.start()
                if 'find' in cmd:
                    found = False
                    obj_name = 'none'
                    for i in cmd:
                        if i in labels:
                            found = True
                            obj_name = i
```

```
                    break
                cmd.clear()
                if obj_name == 'none':
                    speak('cant detect object')
                else:
#                       video.release()
#                       cv2.destroyAllWindows()
                    temp = 'finding '+obj_name
#                       speak(temp)
                    pr = threading.Thread(target=speak,args=(temp,))
                    pr.start()
                    condition = True
                    start_detection(True,obj_name,True)
                    cmd.clear()
            try:
                os.remove(name)
            except:
                pass
        if cv2.waitKey(1) == ord('q') or 'stop' in cmd:
            flag = False
            cmd.clear()
            break
    video.release()
    cv2.destroyAllWindows()
else:
    while True:
        ret , frame = video.read()
        if ret == True:
            count += 1
            size = (416,416)
            frame = cv2.resize(frame,size)
```

```
            if count % 48 == 0:
                name = 'frame {} in {} sec.png'.format((count//5),count//32)
                cv2.imwrite(name,frame)
                image_detection(name,obj_name)
#                 image_detection('testing.png',obj_name)
                t = threading.Thread(target=mic)
                t.start()
                os.remove(name)
        if cv2.waitKey(1) == ord('q') or 'stop' in cmd:
            flag = False
            cmd.clear()
            if start_find:
                flag = True
            break
    video.release()
    cv2.destroyAllWindows()
  return condition
```

6.2.2.3: Code for both finding and detection

```
def image_detection(photo_filename,obj_name='none'):
  input_w, input_h = 416, 416
  input_w, input_h = 416, 416
  image, image_w, image_h = load_image_pixels(photo_filename, (input_w, input_h))
  yhat = model.predict(image)
  anchors = [[116,90, 156,198, 373,326], [30,61, 62,45, 59,119], [10,13, 16,30, 33,23]]
  class_threshold = 0.6
  boxes = list()
  for i in range(len(yhat)):
    boxes += decode_netout(yhat[i][0], anchors[i], class_threshold, input_h, input_w)
  correct_yolo_boxes(boxes, image_h, image_w, input_h, input_w)
```

```
    do_nms(boxes, 0.5)
    labels = ["person", "bicycle", "car", "motorbike", "aeroplane", "bus", "train", "truck",
    "boat", "traffic light", "fire hydrant", "stop sign", "parking meter", "bench","bird",
"cat", "dog", "horse", "sheep", "cow", "elephant", "bear", "zebra", "giraffe,"backpack",
"umbrella", "handbag", "tie", "suitcase", "frisbee", "skis", "snowboard","sports ball",
"kite", "baseball bat", "baseball glove", "skateboard", "surfboard","tennis racket",
"bottle", "wine glass", "cup", "fork", "knife", "spoon", "bowl", "banana","apple",
"sandwich", "orange", "broccoli", "carrot", "hot dog", "pizza", "donut", "cake","chair",
"sofa", "pottedplant", "bed", "diningtable", "toilet", "tvmonitor", "laptop",
"mouse","remote", "keyboard", "cell phone", "microwave", "oven", "toaster", "sink",
"refrigerator","book", "clock", "vase", "scissors", "teddy bear", "hair drier", "toothbrush"]
    v_boxes, v_labels, v_scores = get_boxes(boxes, labels, class_threshold)
    total_detection = len(v_labels)
    r_boxes = []
    r_labels = []
    r_scores = []
    detection = False
    if obj_name != 'none':
        detection = True
        for i in range(total_detection):
            if v_labels[i] == obj_name:
                r_boxes.append(v_boxes[i])
                r_labels.append(v_labels[i])
                r_scores.append(v_scores[i])
    else:
        r_boxes = v_boxes.copy()
        r_labels = v_labels.copy()
        r_scores = v_scores.copy()
    cvdraw(photo_filename, r_boxes, r_labels, r_scores,detection,obj_name)
```

6.2.2.4:Bounding Boxes using CV2

```
def cvdraw(photo_filename, v_boxes, v_labels, v_scores,detection,obj_name):
img=cv2.imread(photo_filename)
direction = ''
height,width,channels = img.shape
image_size = width
ratio = '1:2:1'
ratio_list = list(map(int,ratio.split(':')))
parts = sum(ratio_list)
one_part = image_size // parts
l = ratio_list[0] * one_part
f = ratio_list[1] * one_part
r =  ratio_list[2] * one_part
display = {
'left'  : [0 , l],
'front' : [l + 1 , f + l] ,
'right' : [f + l + 1 , image_size]
}
view = {
    'left' :dict(),
    'front':dict(),
    'right':dict()
}
count = 0
op = []
for i in range(len(v_boxes)):
    count += 1
    font = cv2.FONT_HERSHEY_SIMPLEX
    org = (50, 50)
    fontScale = 0.4
    color = (0, 0, 0)
```

```
        thickness = 1
        x = v_boxes[i].xmin
        y = v_boxes[i].ymin
        w = v_boxes[i].xmax
        h = v_boxes[i].ymax
        op.append([x,y,w])

        check(x,y,w,view,display,v_labels[i],real_width,ref_width)

        confi = str(round(v_scores[i],2)/100)
        confi = confi[0:4]
        text_size, _ = cv2.getTextSize(v_labels[i] + '  ' +confi, font, fontScale,thickness)
        text_w, text_h = text_size
        cv2.rectangle(img, (x,y) , (x + text_w + 5, y - text_h - 5), (0,255,0) , -1)
        cv2.rectangle(img, (x, y), (w, h),color=(0, 255, 0), thickness=2)
        cv2.putText(img, v_labels[i] + '  ' +confi,(x+2,y-2), font, fontScale, color, thickness,
cv2.LINE_AA)
    font = cv2.FONT_HERSHEY_SIMPLEX
    org = (50, 50)
    fontScale = 0.3
    color = (0, 0, 0)
    thickness = 1
    if detection:
        temp = 'Finding '+obj_name+' ...'
        text_size, _ = cv2.getTextSize(temp + '  ', font, fontScale,thickness)
        text_w, text_h = text_size
        cv2.rectangle(img, (5,15) , (5 + text_w + 5, 15 - text_h - 5), (0,255,0) , -1)
        cv2.putText(img,temp,(5+2,15-2),font, fontScale, color, thickness, cv2.LINE_AA)
    else:
        temp = 'Detecting ...'
        text_size, _ = cv2.getTextSize(temp + '  ', font, fontScale,thickness)
```

```
        text_w, text_h = text_size
        cv2.rectangle(img, (5,15) , (5 + text_w + 5, 15 - text_h - 5), (0,255,0) , -1)
        cv2.putText(img,temp,(5+2,15-2),font, fontScale, color, thickness, cv2.LINE_AA)
    cv2.imshow(temp,img)
    if len(v_boxes) != 0:
        p2 = Thread(target = talk , args = (view,))
        p2.start()
    elif detection == True and len(v_boxes) == 0:
        speak('No objects found')
    else:
        speak('You can walk freely now')
```

6.2.3. Code for Direction

```
 def check(x,y,w,view,display,label,real_width,ref_width):
        mid_point = x + w // 2
         if (x + w) < display['left'][1] :
             if label not in view['left'].keys():
                 view['left'][label] = []

view['left'][label].append(round(dis(real_width[label],ref_width[label],w)*10,2))
        return
    if x > display['right'][0]:
        if label not in view['right'].keys():
            view['right'][label] = []
        view['right'][label].append(round(dis(real_width[label],ref_width[label],w)*10,2))
        return
    for k,v in display.items():
        if v[0] <= mid_point and mid_point <= v[1]:
             if label not in view[k].keys():
                 view[k][label] = []
```

```
        view[k][label].append(round(dis(real_width[label],ref_width[label],w)*10,2))
        return
if mid_point < display['left'][0]:
    if label not in view['left'].keys():
        view['left'][label] = []
   view['left'][label].append(round(dis(real_width[label],ref_width[label],w)*10,2))
   return
if label not in view['right'].keys():
    view['right'][label] = []
view['right'][label].append(round(dis(real_width[label],ref_width[label],w)*10,2))
return
```

6.2.3.1. Direction Parameters

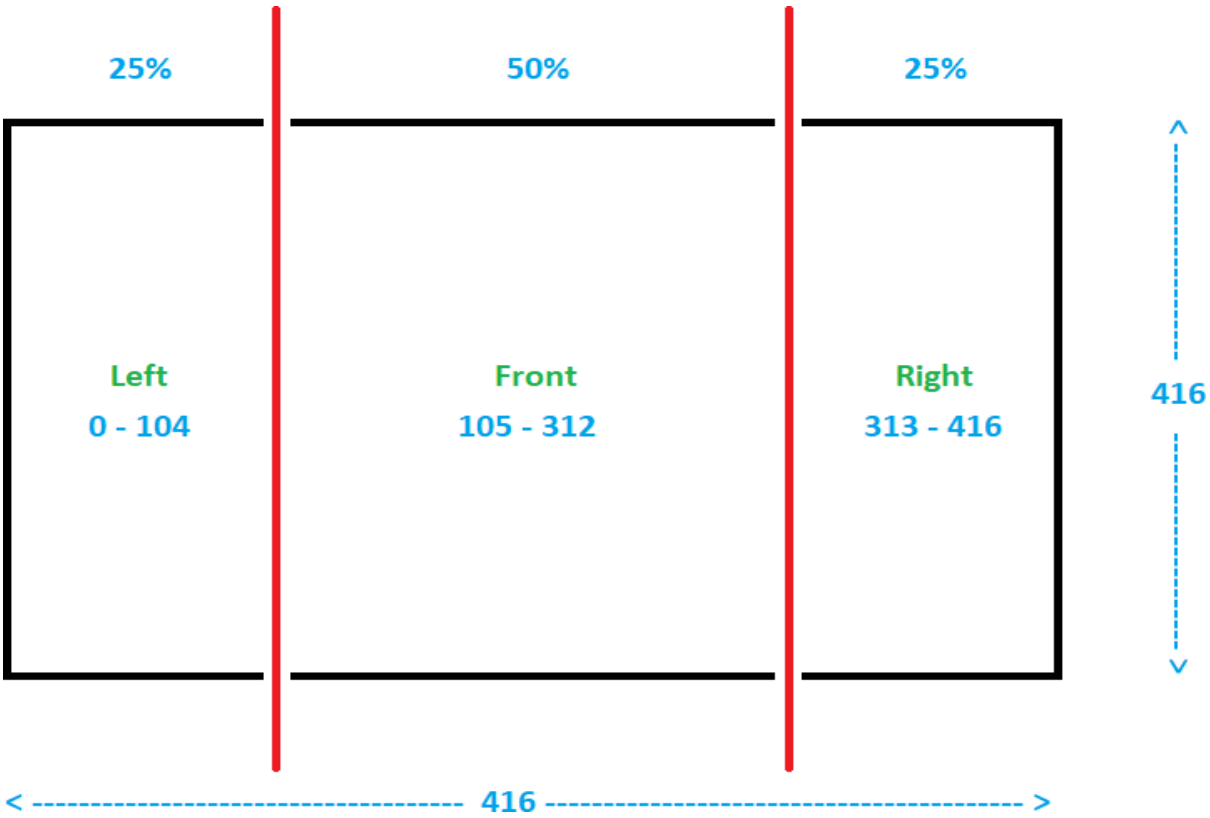
x = x-axis y = y-axis

w = width of object label = object name

real_width = calculated distance of objects

ref_width = reference image distance of objects

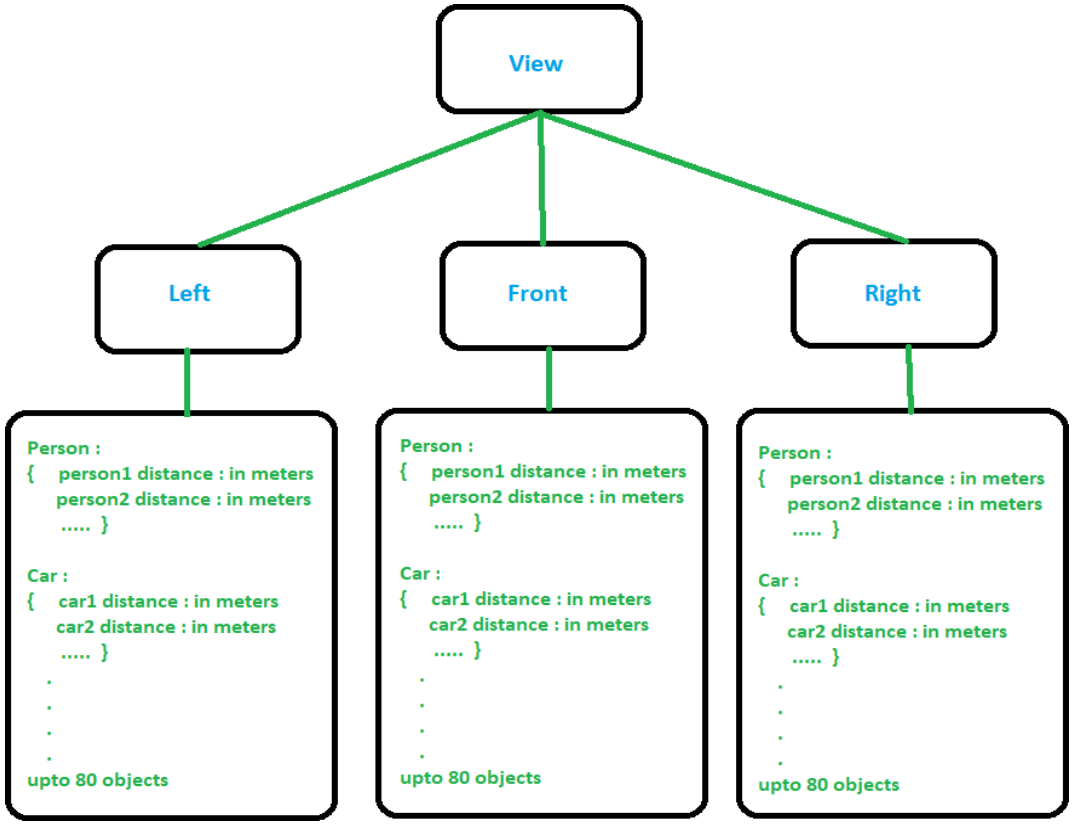
DISPLAY



Direction	Start	End
Left	0	104
Front	105	312
Right	313	416

6.2.3.2. To display Direction

View



6.2.3.3 Logical view of Direction for an object

6.2.4. Code For Distance

```
def dis(real_width,ref_width,width):
    known_dis = 45
    focal_length = (known_dis * real_width ) / ref_width
    distance = (focal_length * real_width ) / width
    return distance
```

Distance Parameters

real_width = object real width

ref_width = detected object reference

image width width = detected object

width

6.2.5.code For voice output

```
def speak(txt):
 speak = p.init()
 speak.say(txt)
 speak.runAndWait()
def talk(view):
 for k,v in view.items():
   if len(v) != 0:
     for k1,v1 in v.items():
       if k == 'front':
         if len(v1) == 1:
        txt = 'one ' + k1 + ' are in ' + k + ' of you ' + str(min(v1)) + ' meters'
           speak(txt)
       else:
       txt = str(len(v1)) + ' ' + k1 + 's are in ' + k + ' of you '  + str(min(v1)) + ' meters'
           speak(txt)
       else:
         if len(v1) == 1:
       txt = 'one ' + k1 + ' is in ' + k + ' side'+ ' from you '  + str(min(v1)) + ' meters'
           speak(txt)
       else:
       txt = str(len(v1)) + ' ' + k1 + 's are in ' + k  + ' side'+ ' from you '  + str(min(v1))
+ ' meters'
     speak(txt)
```

7.TESTING

7.1. Testing Strategy

Testing is the process of trying to find faults or flaws in a product. A testing strategy is defined as a set of instructions or an outline of how the testing is done in a software development life cycle.

Black-box testing:

Black box testing the software without any knowledge of the inner workings of the structure of the module that is being tested. It is also known as a behavioral system. black box testing focuses on the functional requirements of the software. Black box testing can be applied with different such as graph-based testing methods, equivalence partitioning, boundary value analysis, and orthogonal array testing. The test of black box test provides inputs and responds to the outputs without knowing how the software works.

White-box testing:

White box testing is sometimes also called glass-box testing or open-box testing or code-box testing or clear-box testing. It is a testing in which the tester has prior knowledge of the inner workings, and purpose of the software. It is used to test the areas which are not reached from of black box testing level. In this method, testing is based on coverage of code statements, branches, paths, or conditions.

Black-box vs White-box:

Black box testing is considered level testing as it focuses on the behavior of the software whereas white-box testing is considered low-level testing. Black-box testing is not a time-consuming process whereas white-box testing is a time-consuming process.

7.2. Test Cases

7.2.1. Code for Object detection in live video

```
def start_detection(finding = False,obj_name = 'none',start_find = False):
    video = cv2.VideoCapture(0)
    # video = cv2.VideoCapture('sample.mp4')
    count = 0
    condition = False
    global cmd,flag
    flag = True
    cmd = []
    if finding == False:
        while True:
            ret , frame = video.read()
            if ret == True:
                count += 1
                size = (416,416)
                frame = cv2.resize(frame,size)
                if count % 48 == 0:
                    name = 'frame {} in {} sec.png'.format((count//5),count//32)
                    cv2.imwrite(name,frame)
                    print('Entered start_detection function .....')
#                     image_detection('testing.png')
                    image_detection(name)
                    t = threading.Thread(target=mic)
                    t.start()
                    if 'find' in cmd:
                        found = False
                        obj_name = 'none'
                        for i in cmd:
                            if i in labels:
```

```
                found = True
                obj_name = i
                break
            cmd.clear()
            if obj_name == 'none':
                speak('cant detect object')
            else:
                temp = 'finding '+obj_name
                pr = threading.Thread(target=speak,args=(temp,))
                pr.start()
                condition = True
                start_detection(True,obj_name,True)
                cmd.clear()
        try:
            os.remove(name)
        except:
            pass
    if cv2.waitKey(1) == ord('q') or 'stop' in cmd:
        flag = False
        cmd.clear()
        break
video.release()
cv2.destroyAllWindows()
else:
    while True:
        ret , frame = video.read()
        if ret == True:
            count += 1
            size = (416,416)
            frame = cv2.resize(frame,size)
            if count % 48 == 0:
```

```
                name = 'frame {} in {} sec.png'.format((count//5),count//32)
                cv2.imwrite(name,frame)
                image_detection(name,obj_name)
#                 image_detection('testing.png',obj_name)
                t = threading.Thread(target=mic)
                t.start()
                os.remove(name)
            if cv2.waitKey(1) == ord('q') or 'stop' in cmd:
                flag = False
                cmd.clear()
                if start_find:
                    flag = True
                break
        video.release()
        cv2.destroyAllWindows()
    return condition
```

7.2.2. Output for object detection in live video

7.2.2. Output for object detection in live video

7.2.3 Code for finding a object in live video

```
def start_detection(finding = False,obj_name = 'none',start_find = False):
    video = cv2.VideoCapture(0)
    # video = cv2.VideoCapture('sample.mp4')
    count = 0
    condition = False
    global cmd,flag
    flag = True
    cmd = []
    if finding == False:
        while True:
            ret , frame = video.read()
            if ret == True:
                count += 1
                size = (416,416)
                frame = cv2.resize(frame,size)
                if count % 48 == 0:
                    name = 'frame {} in {} sec.png'.format((count//5),count//32)
                    cv2.imwrite(name,frame)
                    print('Entered start_detection function .....')
#                     image_detection('testing.png')
                    image_detection(name)
                    t = threading.Thread(target=mic)
                    t.start()
                    if 'find' in cmd:
                        found = False
                        obj_name = 'none'
                        for i in cmd:
                            if i in labels:
                                found = True
```

```
                obj_name = i
                break
            cmd.clear()
            if obj_name == 'none':
                speak('cant detect object')
            else:
                temp = 'finding '+obj_name
                pr = threading.Thread(target=speak,args=(temp,))
                pr.start()
                condition = True
                start_detection(True,obj_name,True)
                cmd.clear()
        try:
            os.remove(name)
        except:
            pass
    if cv2.waitKey(1) == ord('q') or 'stop' in cmd:
        flag = False
        cmd.clear()
        break
video.release()
cv2.destroyAllWindows()
else:
    while True:
        ret , frame = video.read()
        if ret == True:
            count += 1
            size = (416,416)
            frame = cv2.resize(frame,size)
            if count % 48 == 0:
                name = 'frame {} in {} sec.png'.format((count//5),count//32)
```

```
                cv2.imwrite(name,frame)
                image_detection(name,obj_name)
#                 image_detection('testing.png',obj_name)
                t = threading.Thread(target=mic)
                t.start()
                os.remove(name)
            if cv2.waitKey(1) == ord('q') or 'stop' in cmd:
                flag = False
                cmd.clear()
                if start_find:
                    flag = True
                break
        video.release()
        cv2.destroyAllWindows()
    return condition
```

7.2.4 Output for finding a object in live video

7.2.4 Output for finding a object in live video

8.RESULT

Fig: 8.1. Output for detecting objects

Fig 8.2 Output for finding objects

9. CONCLUSION

9.1. Conclusion

Object detection works and the influence of artificial intelligence help visually impaired people as an assistive system to identify objects and recognize the physical world. The software can be embedded in the application that is used by visually challenged people.

9.2. Future work

- **Navigation**

Navigation can be to our model, so the blind can reach their destination by using the audio from the navigator along with the audio output of distance and direction of object detection that helps the blind to reach the destination safely.

- **Self-training:**

Apart from the 80 objects detection, we can take the custom inputs for object detection in daily life and self-train through our model to get the respective outputs for every daily life object.

REFERENCES & BIBILOGRAPHY

[1] HAIDI ZHU, XIN YAN , HONGYING TANG , YUCHAO CHANG , BAOQING LI , XIAOBING YUAN –" Moving Object Detection With Deep CNNs"-IEEE ,Open Access journal,2020

https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=8988276

[2] Ms. P Devika, S P Jeswanth, Billu Nagamani, T Anuja Chowdary, Madhan Kaveripakam, Nama Chandu-"Object Detection and Recognition using TensorFlow for Blind People"- International Research Journal of Modernization in Engineering Technology andScience2022

https://www.irjmets.com/uploadedfiles/paper/issue_3_march_2022/20216/final/fin_irjmets164830849 0.pdf

[3] - Jemai Bornia; Ali Frihida; Christophe Claramunt "Detecting objects and people and tracking movements in a video using TensorFlow and deep learning"-IEEE Conference paper

https://ieeexplore.ieee.org/document/9318253

[4]- Ashwani Kumar; S S Sai Satyanarayana Reddy; Vivek Kulkarni-" An Object Detection Technique for Blind People in Real-Time Using Deep Neural Network"-IEEE 2019

https://ieeexplore.ieee.org/abstract/document/8985965

[5]Josh Patterson and Adam Gibson, "Deep Learning - A Practitioner's Approach", O'Reilly, (Text Book) 2017, 507 pages.

[6] Swapnil Bhole and Aniket Dhok, "Deep Learning based Object Detection and Recognition Framework for the Visually-Impaired"
IEEE, Conference: 2020 Fourth International Conference on Computing Methodologies and Communication (ICCMC), Erode, India
https://ieeexplore.ieee.org/abstract/document/9076530

[7] Dola Das, Prottoy Saha and Nawshin Tabassum, "Real Time Distance and Mobility Detection of Obstacles Using a Smart Multisensor Framework for

Visually Impaired People", IEEE, 2020.
https://ieeexplore.ieee.org/document/9230880
[8] K. K. Baseer; M. Jahir Pasha; and D. William Albert; V. Sujatha, "Navigation And Obstacle Detection For Visually Impaired People", IEEE, 2021

https://ieeexplore.ieee.org/document/9673618
[9] Chan-Su Lee; Jae-Ik Lee; Han Eol Seo, "Deep Learning Based Mobile Assistive Device for Visually Impaired People" - 2021 IEEE International Conference on Consumer Electronics- Asia (ICCE-Asia)
https://ieeexplore.ieee.org/document/9641925
[10] Aisyah Widayani; Hendra Kusuma; and Djoko Purwanto, "Visually Impaired Person Detection Using Deep Learning for Dangerous Area Warning System", IEEE, 2022. https://ieeexplore.ieee.org/document/9855268
[11] Tayyip Mert Denizgez; Orçun Kamiloğlu; Seda Kul; and Ahmet Sayar, "Guiding Visually Impaired People to Find an Object by Using Image to Speech over the Smart Phone Cameras", IEEE, 2021.
https://ieeexplore.ieee.org/document/9548122
[12] Heba Najm; Khirallah Elferjani; and Alhaam Alariyibi, "Assisting Blind People Using Object Detection with Vocal Feedback" - 2022 IEEE 2nd International Maghreb Meeting of the Conference on Sciences and Techniques of Automatic Control and Computer Engineering (MI-STA)
https://ieeexplore.ieee.org/document/9837737
[13] Zifeng Wang; Heng Li; Jianpin Chen; Xinyu Chai; and Zhenzhen Zhai, "A Wearable Vision-To-Audio Sensory Substitution System Based on Deep Learning for the Visually Impaired" – 2021 IEEE International Conference on Digital Society and Intelligent Systems (DSInS)

https://ieeexplore.ieee.org/document/9670599
[14] Rajalingappaa Shanmugamani, "Deep Learning for Computer Vision - Expert techniques to train advanced neural networks using TensorFlow and Keras" (TextBook), Packt Publishing, Birmingham, UK, 2018, 310 page

APPENDIX- A

A1. FULL CODE

```
from numpy import expand_dims
from keras.models import load_model
from keras_preprocessing.image import load_img
from keras_preprocessing.image import img_to_array
from keras import losses
from keras import optimizers
from keras import metrics
import numpy as np
from numpy import expand_dims
from keras.models import load_model
from keras_preprocessing.image import load_img
from keras_preprocessing.image import img_to_array
from matplotlib import pyplot
from matplotlib.patches import Rectangle
from numpy import expand_dims
from keras.models import load_model
from keras_preprocessing.image import load_img
from keras_preprocessing.image import img_to_array
from PIL import Image, ImageFont, ImageDraw
import cv2
import pyttsx3 as p
from threading import Thread

class BoundBox:
def __init__(self, xmin, ymin, xmax, ymax, objness = None, classes = None):
self.xmin = xmin
self.ymin = ymin
self.xmax = xmax
```

```
self.ymax = ymax
self.objness = objness
self.classes = classes
self.label = -1
self.score = -1
def get_label(self):
if self.label == -1:
self.label = np.argmax(self.classes)
return self.label
def get_score(self):
if self.score == -1:
self.score = self.classes[self.get_label()]
return self.score
def _sigmoid(x):
return 1. / (1. + np.exp(-x))
def decode_netout(netout, anchors, obj_thresh, net_h, net_w):
grid_h, grid_w = netout.shape[:2]
nb_box = 3
netout = netout.reshape((grid_h, grid_w, nb_box, -1))
nb_class = netout.shape[-1] - 5
boxes = []
netout[..., :2]  = _sigmoid(netout[..., :2])
netout[..., 4:]  = _sigmoid(netout[..., 4:])
netout[..., 5:]  = netout[..., 4][..., np.newaxis] * netout[..., 5:]
netout[..., 5:] *= netout[..., 5:] > obj_thresh
for i in range(grid_h*grid_w):
row = i / grid_w
col = i % grid_w
for b in range(nb_box):
objectness = netout[int(row)][int(col)][b][4]
if(objectness.all() <= obj_thresh): continue
```

```
x, y, w, h = netout[int(row)][int(col)][b][:4]
x = (col + x) / grid_w
y = (row + y) / grid_h
w = anchors[2 * b + 0] * np.exp(w) / net_w
h = anchors[2 * b + 1] * np.exp(h) / net_h
classes = netout[int(row)][col][b][5:]
box = BoundBox(x-w/2, y-h/2, x+w/2, y+h/2, objectness, classes)
boxes.append(box)
return boxes
def correct_yolo_boxes(boxes, image_h, image_w, net_h, net_w):
new_w, new_h = net_w, net_h
for i in range(len(boxes)):
x_offset, x_scale = (net_w - new_w)/2./net_w, float(new_w)/net_w
y_offset, y_scale = (net_h - new_h)/2./net_h, float(new_h)/net_h
boxes[i].xmin = int((boxes[i].xmin - x_offset) / x_scale * image_w)
boxes[i].xmax = int((boxes[i].xmax - x_offset) / x_scale * image_w)
boxes[i].ymin = int((boxes[i].ymin - y_offset) / y_scale * image_h)
boxes[i].ymax = int((boxes[i].ymax - y_offset) / y_scale * image_h)
def _interval_overlap(interval_a, interval_b):
x1, x2 = interval_a
x3, x4 = interval_b
if x3 < x1:
if x4 < x1:
return 0
else:
return min(x2,x4) - x1
else:
if x2 < x3:
return 0
else:
return min(x2,x4) - x3
```

```
def bbox_iou(box1, box2):
intersect_w = _interval_overlap([box1.xmin, box1.xmax], [box2.xmin, box2.xmax])
intersect_h = _interval_overlap([box1.ymin, box1.ymax], [box2.ymin, box2.ymax])
intersect = intersect_w * intersect_h
w1, h1 = box1.xmax-box1.xmin, box1.ymax-box1.ymin
w2, h2 = box2.xmax-box2.xmin, box2.ymax-box2.ymin
union = w1*h1 + w2*h2 - intersect
return float(intersect) / union
def do_nms(boxes, nms_thresh):
if len(boxes) > 0:
nb_class = len(boxes[0].classes)
else:
return
for c in range(nb_class):
sorted_indices = np.argsort([-box.classes[c] for box in boxes])
for i in range(len(sorted_indices)):
index_i = sorted_indices[i]
if boxes[index_i].classes[c] == 0: continue
for j in range(i+1, len(sorted_indices)):
index_j = sorted_indices[j]
if bbox_iou(boxes[index_i], boxes[index_j]) >= nms_thresh:
boxes[index_j].classes[c] = 0
# load and prepare an image
def load_image_pixels(filename, shape):
image = load_img(filename)
width, height = image.size
image = load_img(filename, target_size=shape)
image = img_to_array(image)
image = image.astype('float32')
image /= 255.0
image = expand_dims(image, 0)
```

```
return image, width, height
def get_boxes(boxes, labels, thresh):
v_boxes, v_labels, v_scores = list(), list(), list()
for box in boxes:
for i in range(len(labels)):
if box.classes[i] > thresh:
v_boxes.append(box)
v_labels.append(labels[i])
v_scores.append(box.classes[i]*100)
return v_boxes, v_labels, v_scores
def dis(real_width,ref_width,width):
    known_dis = 45
    focal_length = (known_dis * real_width ) / ref_width
    distance = (focal_length * real_width ) / width
    return distance
# ------------------------------------------- voice output ---------------------------------------------
-
def speak(txt):
    speak = p.init()
    speak.say(txt)
    speak.runAndWait()
def talk(view):
    for k,v in view.items():
        if len(v) != 0:
            for k1,v1 in v.items():
                if k == 'front':
                    if len(v1) == 1:
                        txt = 'one ' + k1 + ' are in ' + k + ' of you ' + str(min(v1)) + ' meters'
                        speak(txt)
                    else:
                  txt = str(len(v1)) + ' ' + k1 + 's are in ' + k + ' of you '  + str(min(v1)) + ' meters'
```

```
                speak(txt)
            else:
                if len(v1) == 1:
            txt = 'one ' + k1 + ' is in ' + k + ' side'+ ' from you '  + str(min(v1)) + ' meters'
                speak(txt)
                else:
            txt = str(len(v1)) + ' ' + k1 + 's are in ' + k  + ' side'+ ' from you '  + str(min(v1))
+ ' meters'
                speak(txt)
# ---------------------------------------- direction dictionary ----------------------------------------
def check(x,y,w,view,display,label,real_width,ref_width):
    mid_point = x + w // 2
    if (x + w) < display['left'][1] :
        if label not in view['left'].keys():
            view['left'][label] = []
        view['left'][label].append(round(dis(real_width[label],ref_width[label],w)*10,2))
        return
    if x > display['right'][0]:
        if label not in view['right'].keys():
            view['right'][label] = []
        view['right'][label].append(round(dis(real_width[label],ref_width[label],w)*10,2))
        return
    for k,v in display.items():
        if v[0] <= mid_point and mid_point <= v[1]:
            if label not in view[k].keys():
                view[k][label] = []
            view[k][label].append(round(dis(real_width[label],ref_width[label],w)*10,2))
            return
    if mid_point < display['left'][0]:
        if label not in view['left'].keys():
            view['left'][label] = []
```

```
        view['left'][label].append(round(dis(real_width[label],ref_width[label],w)*10,2))
        return
    if label not in view['right'].keys():
        view['right'][label] = []
    view['right'][label].append(round(dis(real_width[label],ref_width[label],w)*10,2))
    return
# ---------------------------------------- distance estimation varaibles ------------------------------
---------
real_width = {"person" : 16, "bicycle" : 66, "car" : 109, "motorbike" : 57, "aeroplane" :
590, "bus": 472 , "train": 529 , "truck" : 496,"boat": 120 , "traffic light" : 11,  "fire
hydrant": 12  , "stop sign" : 20 , "parking meter": 177 , "bench":48 ,"bird":9 , "cat":15 ,
"dog":25 , "horse": 35 , "sheep": 27 , "cow": 35 , "elephant": 145 , "bear": 96, "zebra":96 ,
"giraffe":192 ,"backpack":12 , "umbrella":51 , "handbag":9 , "tie": 3 , "suitcase":20 ,
"frisbee":3937  , "skis":70 , "snowboard":63 ,"sports ball":9 , "kite":9 , "baseball bat":34 ,
"baseball glove":13 , "skateboard":9 , "surfboard":77 ,"tennis racket":27 , "bottle":3 ,
"wine glass":3 , "cup":4 , "fork":6 , "knife":8 , "spoon":6 , "bowl":5 , "banana":6
,"apple":3 , "sandwich":4 , "orange":3 , "broccoli":4 , "carrot":5 , "hot dog":6 , "pizza":8 ,
"donut":5 , "cake":10 ,"chair":17 , "sofa":77 , "pottedplant":9 , "bed":85 ,
"diningtable":50 , "toilet":16 , "tvmonitor":32 , "laptop":15 , "mouse":4 ,"remote":6 ,
"keyboard":15 , "cell phone":5 , "microwave":14 , "oven":17 , "toaster":8 , "sink":19 ,
"refrigerator":23 ,"book":11 , "clock":12 , "vase":9 , "scissors":7 , "teddy bear":15 , "hair
drier":12 , "toothbrush":7 }
ref_width =   {'person': 179, 'bicycle': 387, 'car': 334, 'motorbike': 395, 'aeroplane': 296,
'bus': 283, 'train': 290, 'truck': 329, 'boat': 230,'traffic light': 349, 'fire hydrant': 389, 'stop
sign': 191, 'parking meter': 382, 'bench': 342, 'bird': 315, 'cat': 341, 'dog': 242, 'horse': 386,
'sheep': 377, 'cow': 339, 'elephant': 329, 'bear': 270, 'zebra': 385,'giraffe': 211, 'backpack':
335, 'umbrella': 387, 'handbag': 285, 'tie': 374,'suitcase': 170, 'frisbee': 303, 'skis': 174,
'snowboard': 128, 'sports ball': 361, 'kite': 338, 'baseball bat': 300,'baseball glove': 160,
'skateboard': 378, 'surfboard': 229, 'tennis racket': 417, 'bottle': 70, 'wine glass': 256,
'cup': 255, 'fork': 315, 'knife': 395, 'spoon': 379, 'bowl': 376, 'banana': 356, 'apple': 272,
'sandwich': 322,'orange': 295, 'broccoli': 261, 'carrot': 243, 'hot dog': 379, 'pizza': 388,
```

```
'donut': 342, 'cake': 378,'chair': 258, 'sofa': 336, 'pottedplant': 407, 'bed': 394, 'diningtable':
350, 'toilet': 292,'tvmonitor': 375, 'laptop': 378, 'mouse': 275, 'remote': 434, 'keyboard':
180, 'cell phone': 256,'microwave': 367, 'oven': 417, 'toaster': 136, 'sink': 120,
'refrigerator': 257, 'book': 462,'clock': 299, 'vase': 157, 'scissors': 379, 'teddy bear': 273,
'hair drier': 109, 'toothbrush': 412}
# ---------------------------------------- drawing rectangle in cv2 ------------------------------------
----
def cvdraw(photo_filename, v_boxes, v_labels, v_scores,detection,obj_name):
    img=cv2.imread(photo_filename)
    direction = ''
    height,width,channels = img.shape
    image_size = width
    ratio = '1:2:1'
    ratio_list = list(map(int,ratio.split(':')))
    parts = sum(ratio_list)
    one_part = image_size // parts
    l = ratio_list[0] * one_part
    f = ratio_list[1] * one_part
    r =  ratio_list[2] * one_part
    display = {
    'left'  : [0 , l],
    'front' : [l + 1 , f + l] ,
    'right' : [f + l + 1 , image_size]
    }
    view = {
        'left' :dict(),
        'front':dict(),
        'right':dict()
    }

    count = 0
```

```
    op = []
    for i in range(len(v_boxes)):
        count += 1
        font = cv2.FONT_HERSHEY_SIMPLEX
        org = (50, 50)
        fontScale = 0.4
        color = (0, 0, 0)
        thickness = 1
        x = v_boxes[i].xmin
        y = v_boxes[i].ymin
        w = v_boxes[i].xmax
        h = v_boxes[i].ymax
        op.append([x,y,w])

        check(x,y,w,view,display,v_labels[i],real_width,ref_width)

        confi = str(round(v_scores[i],2)/100)
        confi = confi[0:4]
        text_size, _ = cv2.getTextSize(v_labels[i] + '  ' +confi, font, fontScale,thickness)
        text_w, text_h = text_size
        cv2.rectangle(img, (x,y) , (x + text_w + 5, y - text_h - 5), (0,255,0) , -1)
        cv2.rectangle(img, (x, y), (w, h),color=(0, 255, 0), thickness=2)
        cv2.putText(img, v_labels[i] + '  ' +confi,(x+2,y-2), font, fontScale, color, thickness,
cv2.LINE_AA)
    font = cv2.FONT_HERSHEY_SIMPLEX
    org = (50, 50)
    fontScale = 0.3
    color = (0, 0, 0)
    thickness = 1
    if detection:
        temp = 'Finding '+obj_name+' ...'
```

```
        text_size, _ = cv2.getTextSize(temp + '  ', font, fontScale,thickness)
        text_w, text_h = text_size
        cv2.rectangle(img, (5,15) , (5 + text_w + 5, 15 - text_h - 5), (0,255,0) , -1)
        cv2.putText(img,temp,(5+2,15-2),font, fontScale, color, thickness, cv2.LINE_AA)
    else:
        temp = 'Detecting ...'
        text_size, _ = cv2.getTextSize(temp + '  ', font, fontScale,thickness)
        text_w, text_h = text_size
        cv2.rectangle(img, (5,15) , (5 + text_w + 5, 15 - text_h - 5), (0,255,0) , -1)
        cv2.putText(img,temp,(5+2,15-2),font, fontScale, color, thickness, cv2.LINE_AA)

    cv2.imshow(temp,img)
    if len(v_boxes) != 0:
        p2 = Thread(target = talk , args = (view,))
        p2.start()
    elif detection == True and len(v_boxes) == 0:
        speak('No objects found')
    else:
        speak('You can walk freely now')
labels = ["person", "bicycle", "car", "motorbike", "aeroplane", "bus", "train", "truck",
 "boat", "traffic light", "fire hydrant", "stop sign", "parking meter", "bench","bird", "cat",
"dog", "horse", "sheep", "cow", "elephant", "bear", "zebra", "giraffe","backpack",
"umbrella", "handbag", "tie", "suitcase", "frisbee", "skis", "snowboard","sports ball",
"kite", "baseball bat", "baseball glove", "skateboard", "surfboard","tennis racket",
"bottle", "wine glass", "cup", "fork", "knife", "spoon", "bowl", "banana","apple",
"sandwich", "orange", "broccoli", "carrot", "hot dog", "pizza", "donut", "cake","chair",
"sofa", "pottedplant", "bed", "diningtable", "toilet", "tvmonitor", "laptop",
"mouse","remote", "keyboard", "cell phone", "microwave", "oven", "toaster", "sink",
"refrigerator","book", "clock", "vase", "scissors", "teddy bear", "hair drier",
"toothbrush"]
```

```
import cv2
model = load_model('model.h5')
model.compile(loss = 'mean_squared_error',optimizer = 'sgd', metrics =
[metrics.categorical_accuracy])
def image_detection(photo_filename,obj_name='none'):
    input_w, input_h = 416, 416
    input_w, input_h = 416, 416
    image, image_w, image_h = load_image_pixels(photo_filename, (input_w, input_h))
    yhat = model.predict(image)
    anchors = [[116,90, 156,198, 373,326], [30,61, 62,45, 59,119], [10,13, 16,30, 33,23]]
    class_threshold = 0.6
    boxes = list()
    for i in range(len(yhat)):
        boxes += decode_netout(yhat[i][0], anchors[i], class_threshold, input_h, input_w)
    correct_yolo_boxes(boxes, image_h, image_w, input_h, input_w)
    do_nms(boxes, 0.5)
    global labels
    v_boxes, v_labels, v_scores = get_boxes(boxes, labels, class_threshold)
    total_detection = len(v_labels)
    r_boxes = []
    r_labels = []
    r_scores = []
    detection = False
    if obj_name != 'none':
        detection = True
        for i in range(total_detection):
            if v_labels[i] == obj_name:
                r_boxes.append(v_boxes[i])
                r_labels.append(v_labels[i])
                r_scores.append(v_scores[i])
```

```
    else:
        r_boxes = v_boxes.copy()
        r_labels = v_labels.copy()
        r_scores = v_scores.copy()
    cvdraw(photo_filename, r_boxes, r_labels, r_scores,detection,obj_name)


import os
import cv2
import speech_recognition as sr
import pyttsx3
from threading import Thread
global labels
r = sr.Recognizer()
cmd = ''
flag = True
def mic():
    global cmd,flag
    while flag:
        print('Entered mic function ....')
        try:
            with sr.Microphone() as source2:
                r.adjust_for_ambient_noise(source2, duration=0.2)
                audio2 = r.listen(source2)
                MyText = r.recognize_google(audio2)
                MyText = MyText.lower()
                names = list(MyText.split())
                cmd = names
                print(cmd)
                if 'stop' in cmd:
                    flag = False
                    break
```

```
            print(MyText)
        except sr.RequestError as e:
            print("Could not request results; {0}".format(e))

        except sr.UnknownValueError:
            print("unknown error occurred")
def start_detection(finding = False,obj_name = 'none',start_find = False):
    video = cv2.VideoCapture(0)
    count = 0
    condition = False
    global cmd,flag
    flag = True
    cmd = []
    if finding == False:
        while True:
            ret , frame = video.read()
            if ret == True:
                count += 1
                size = (416,416)
                frame = cv2.resize(frame,size)
                if count % 48 == 0:
                    name = 'frame {} in {} sec.png'.format((count//5),count//32)
                    cv2.imwrite(name,frame)
                    print('Entered start_detection function .....')
                    image_detection(name)
                    t = threading.Thread(target=mic)
                    t.start()
                    if 'find' in cmd:
                        found = False
                        obj_name = 'none'
                        for i in cmd:
```

```
                if i in labels:
                    found = True
                    obj_name = i
                    break
            cmd.clear()
            if obj_name == 'none':
                speak('cant detect object')
            else:
                temp = 'finding '+obj_name
                pr = threading.Thread(target=speak,args=(temp,))
                pr.start()
                condition = True
                start_detection(True,obj_name,True)
                cmd.clear()
        try:
            os.remove(name)
        except:
            pass
    if cv2.waitKey(1) == ord('q') or 'stop' in cmd:
        flag = False
        cmd.clear()
        break
video.release()
cv2.destroyAllWindows()
else:
    while True:
        ret , frame = video.read()
        if ret == True:
            count += 1
            size = (416,416)
            frame = cv2.resize(frame,size)
```

```
            if count % 48 == 0:
                name = 'frame {} in {} sec.png'.format((count//5),count//32)
                cv2.imwrite(name,frame)
                image_detection(name,obj_name)
                t = threading.Thread(target=mic)
                t.start()
                os.remove(name)
        if cv2.waitKey(1) == ord('q') or 'stop' in cmd:
            flag = False
            cmd.clear()
            if start_find:
                flag = True
            break
    video.release()
    cv2.destroyAllWindows()
  return condition
import speech_recognition as sr
import pyttsx3
import threading
r = sr.Recognizer()

global labels
def main():
  while(1):
    print('Entered main function .....')
    try:
        with sr.Microphone() as source2:
            r.adjust_for_ambient_noise(source2, duration=0.2)
            audio2 = r.listen(source2)
            MyText = r.recognize_google(audio2)
            MyText = MyText.lower()
```

```
            names = list(MyText.split())
            print(names)
            print("Did you say ",MyText)
            if 'start' in MyText:
                speak('detecting')
                condition = start_detection()
                if condition:
                    start_detection()
            elif 'find' in MyText:
                found = False
                obj_name = 'none'
                for i in names:
                    if i in labels:
                        found = True
                        obj_name = i
                        break
                if obj_name == 'none':
                    speak('cant detect object')
                else:
                    temp = 'finding '+obj_name
                    speak(temp)
                    start_detection(True,obj_name)
            elif 'stop' in MyText:
                break
        except sr.RequestError as e:
            print("Could not request results; {0}".format(e))
        except sr.UnknownValueError:
            print("unknown error occurred")
main()
```

A2.SCREENSHOTS

Output for detecting objects

Output for finding objects